

异常控制流：异常与进程

计算机系统基础

任课教师：

龚奕利

yiligong@whu.edu.cn

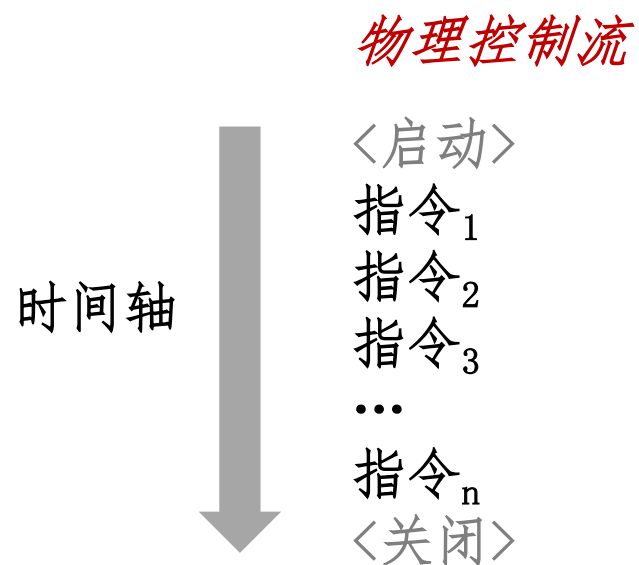
主要内容

- 异常控制流
- 异常
- 进程
- 系统调用
- 进程控制

控制流

■ 处理器只做一件事：

- 从启动到关闭，CPU 仅仅逐条读取并执行（解释）指令序列
- 这一指令序列构成了 CPU 的控制流



改变控制流

- 到目前为止，有两种机制可以改变控制流：

- 跳转和分支
- 调用和返回

响应 **程序状态** 的变化

- 对于构建一个实用的系统来说还不足够：难以响应 **系统状态** 的变化

- 数据从磁盘或网络适配器到达
- 指令发生除零
- 用户按下键盘上的 `Ctrl-C`
- 系统定时器到时

- 系统需要“异常控制流”机制

异常控制流

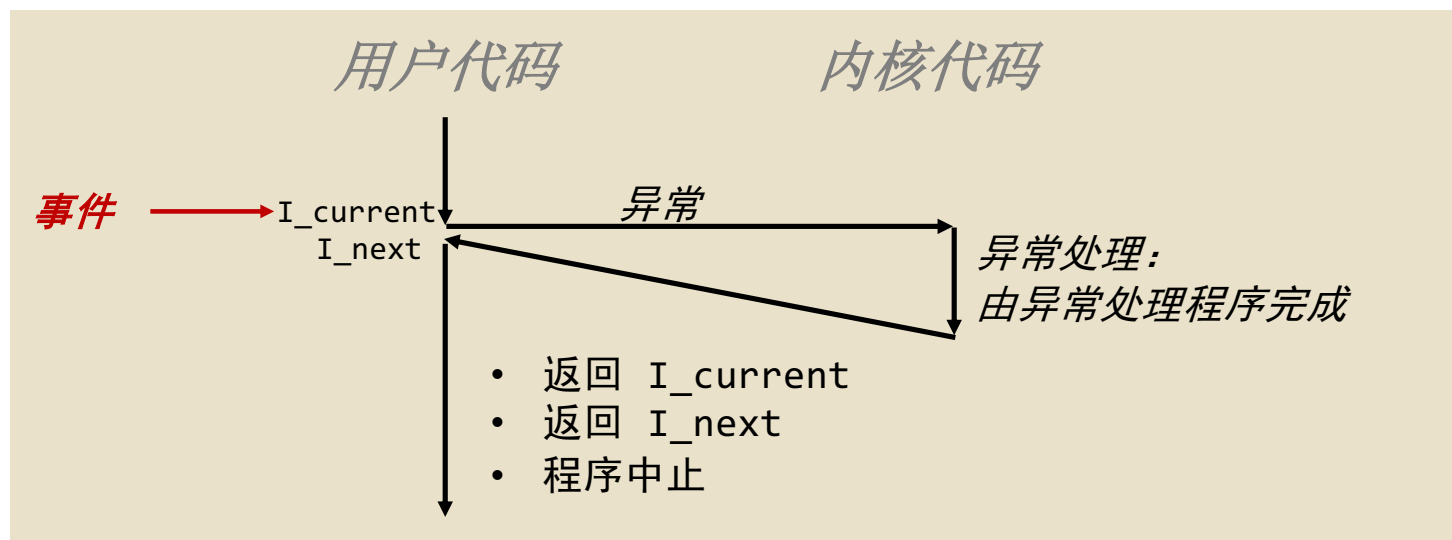
- 发生在计算机系统的各个层次
- 低层次机制
 - 1. 异常
 - 为响应系统事件（即系统状态变化）引发的控制流变化
 - 通过硬件和操作系统软件的结合实现
- 高层次机制
 - 2. 进程上下文切换
 - 通过操作系统软件和硬件定时器实现
 - 3. 信号
 - 通过操作系统软件实现
 - 4. 非本地跳转
 - `setjmp()` and `longjmp()`
 - 由 C 运行时库实现

主要内容

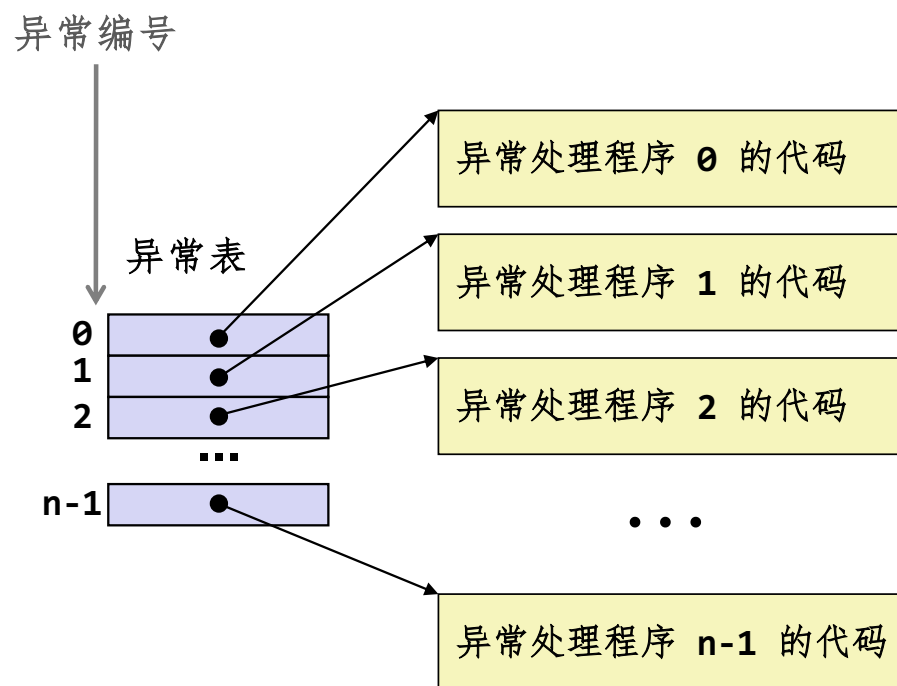
- 异常控制流
- 异常
- 进程
- 系统调用
- 进程控制

异常

- **异常**是指在响应某些事件（即处理器状态变化）时，控制流被转移到操作系统内核的过程
 - 内核是操作系统中常驻内存的部分
 - 事件示例：除零，算术溢出，缺页异常，I/O 请求完成，用户按下 Ctrl-C



异常表



- 每种类型的事件都有一个唯一的异常号 k
- k = 在异常表中的索引（又称中断向量）
- 每次发生异常 k 时，都会调用对应的处理程序 k

异步异常（中断）

■ 由处理器外部事件引发

- 通过向处理器的 *中断引脚*(interrupt pin)发信号来表明
- 处理程序返回到“下一条”指令

■ 例如：

- 定时器中断
 - 每隔几毫秒，外部定时器芯片触发一次中断
 - 内核利用此机制从用户程序中取回控制权
- 外部设备的 I/O 中断
 - 用户按下 **Ctrl-C**
 - 网络数据包到达
 - 磁盘数据到达

同步异常

■ 由执行指令时发生的事件引发

■ 陷阱

- 有意的
- 示例：系统调用、断点陷阱、特殊指令
- 返回控制权到“下一条”指令

■ 故障

- 无意的，但可能是可恢复的
- 示例：缺页异常（可恢复的），保护错误（不可恢复的），浮点异常
- 重新执行导致故障的（“当前”）指令或终止程序

■ 终止

- 无意的，且不可恢复的
- 示例：非法指令、奇偶校验错、硬件检测错误
- 终止当前程序

系统调用

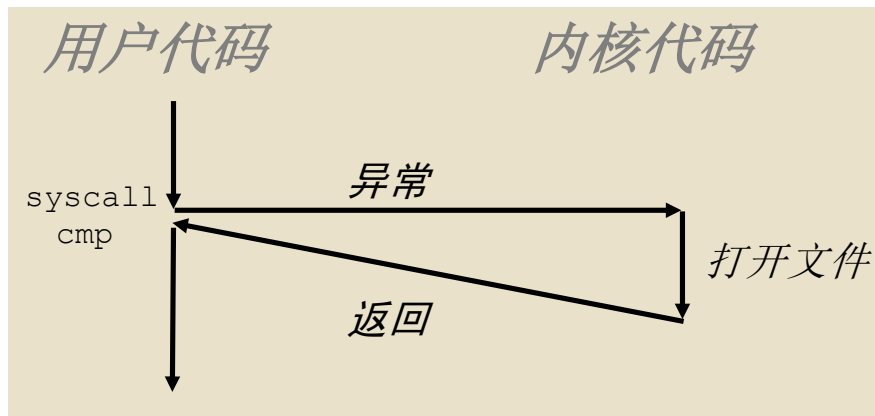
- 每个 **x86-64** 系统调用都有一个唯一的 **ID** 号
- 例如：

编号	名字	描述
0	read	读文件
1	write	写文件
2	open	打开文件
3	close	关闭文件
4	stat	获得文件信息
57	fork	创建进程
59	execve	执行一个程序
60	_exit	终止进程
62	kill	发送信号到一个进程

系统调用示例：打开文件

- 用户调用：`open(filename, options)`
- 调用 `__open` 函数，该函数触发系统调用指令 `syscall`

```
00000000000e5d70 <__open>:  
...  
e5d79:  b8 02 00 00 00      mov  $0x2,%eax  # open is syscall #2  
e5d7e:  0f 05               syscall          # return value in %rax  
e5d80:  48 3d 01 f0 ff ff   cmp  $0xffffffffffffffff001,%rax  
...  
e5dfa:  c3                  retq
```



- `%rax` 包含系统调用号
- `%rdi, %rsi, %rdx, %r10, %r8, %r9` 包含最多6个参数
- 从系统调用返回时，`%rax` 包含返回值
- 负数返回值表明发生了错误，对应于负的 `errno`

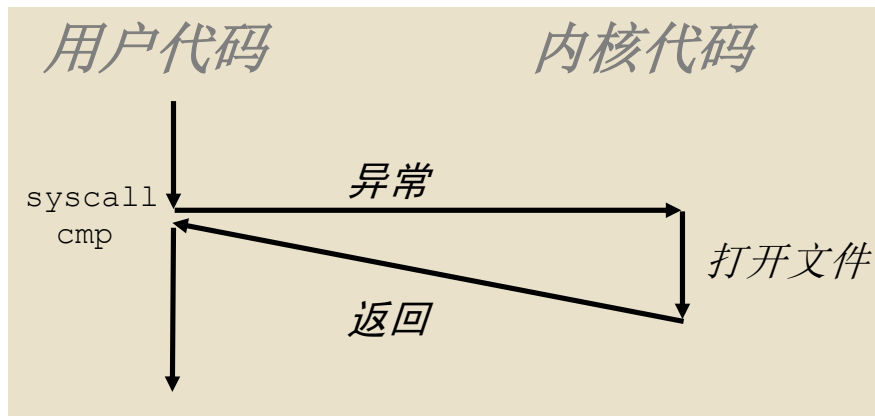
系统调用示例：打开文件

- 用户调用：`open(filename, options)`
- 调用 `__open` 函数，该函数触发系统调用指令 `syscall`

0000000000e5d70 <__open>:

```
...  
e5d79:  b8 02 00 00 00      mov  $0x2,%eax  # open is syscall #2  
e5d7e:  0f 05               syscall         # return value in %rax  
e5d80:  48 3d 01 f0 ff ff   cmp  $0xfffffffffffff001,%rax  
...  
e5dfa:  c3                 retq
```

0xffff ffff ffff f001 -4095，错误号是一个-4095到-1之间的一个负数。

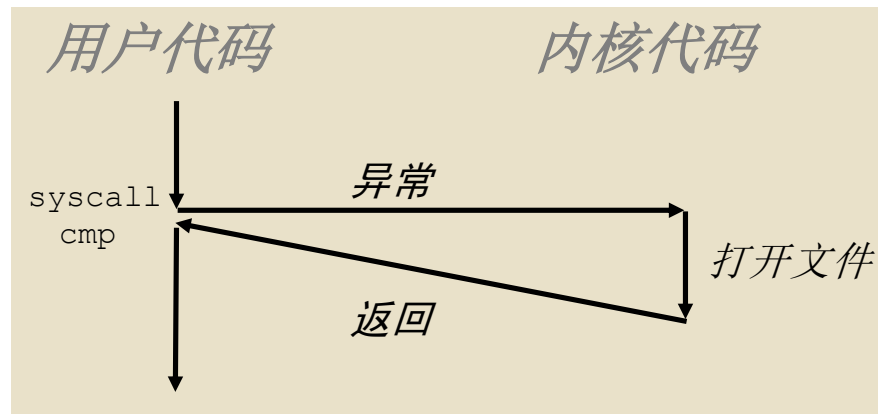


- `%rax` 包含系统调用号
- `%rdi, %rsi, %rdx, %r10, %r8, %r9` 包含最多6个参数
- 从系统调用返回时，`%rax` 包含返回值
- 负数返回值表明发生了错误，对应于负的 `errno`

系统调用示例：打开文件

- 用户调用：`open(filename, options)`
- 调用 `__open` 函数，该函数触发系统调用指令 `syscall`

```
0000000000e5d70 <__open>:  
...  
e5d79:  b8 02 00 00 00      mov  $0x2,%rax  
e5d7e:  0f 05               syscall  
e5d80:  48 3d 01 f0 ff ff    cmp  $0xffff,%rax  
...  
e5dfa:  c3                  retq
```



几乎类似于函数调用

- 控制流的转移
- 返回时执行下一条指令
- 按照调用约定传递参数
- 结果存储于 `%rax`

一个重要的例外！

- 由内核执行
- 拥有不同的权限级别
- 还有其他差异：
 - 例如，“函数”的“地址”存储于 `%rax`
 - 使用 `errno`
 - etc.

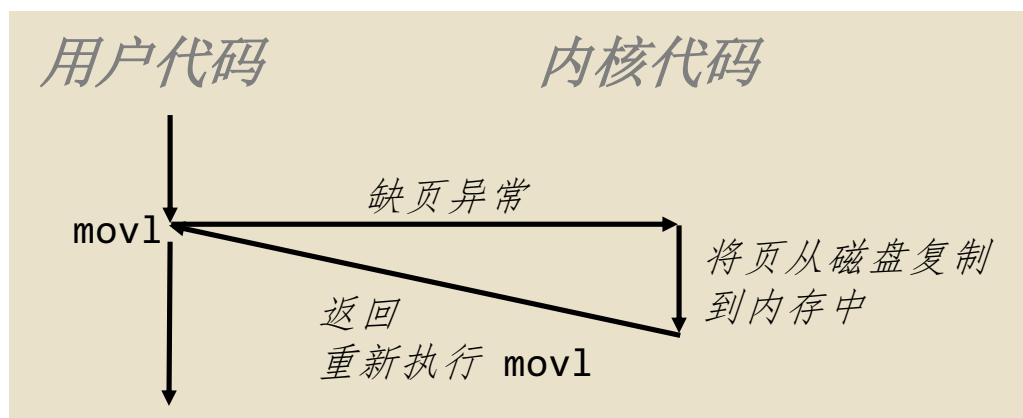
- `%rax` 包含系统调用号
- `%rdi, %rsi, %rdx, %r10, %r8, %r9` 包含最多6个参数
- 从系统调用返回时，`%rax` 包含返回值
- 负数返回值表明发生了错误，对应于负的 `errno`

故障示例：缺页异常

- 用户尝试写入一个内存地址
- 用户内存的该部分（页）当前存储在磁盘上

```
int a[1000];  
main ()  
{  
    a[500] = 13;  
}
```

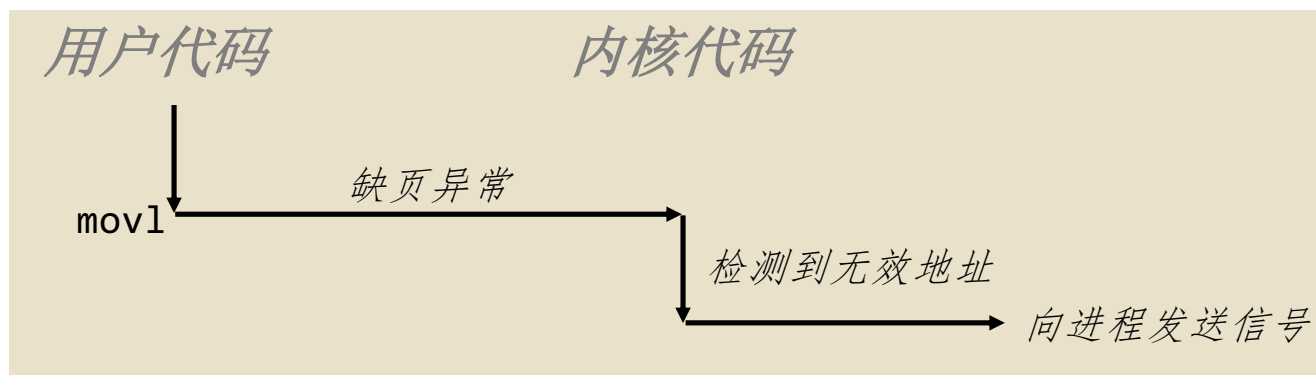
80483b7:	c7 05 10 9d 04 08 0d	movl	\$0xd,0x8049d10
----------	----------------------	------	-----------------



故障示例：无效的内存引用

```
int a[1000];  
main ()  
{  
    a[5000] = 13;  
}
```

80483b7:	c7 05 60 e3 04 08 0d	movl	\$0xd,0x804e360
----------	----------------------	------	-----------------



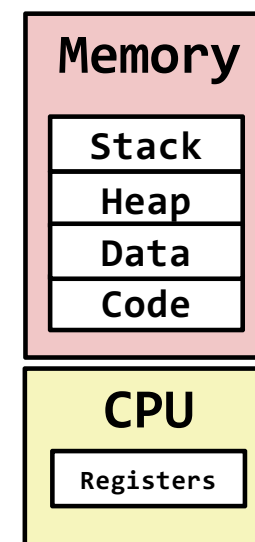
- 向用户进程发送 **SIGSEGV** 信号
- 用户进程退出，报“segmentation fault”错

主要内容

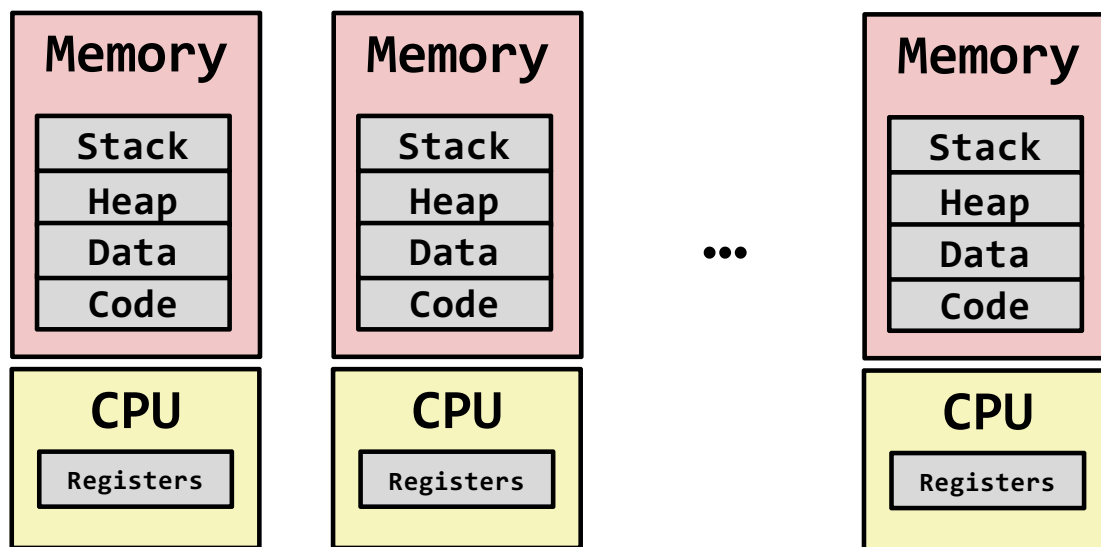
- 异常控制流
- 异常
- 进程
- 系统调用
- 进程控制

进程 (process)

- 定义： **进程** 是一个正在运行的程序的实例
 - 计算机科学中最深刻的思想之一
 - 与“程序”或“处理器 (processor)”不同
- 进程为每个程序提供两个关键抽象：
 - **逻辑控制流**
 - 看起来好像程序独占地使用 CPU
 - 由称为上下文切换的内核机制提供
 - **私有地址空间**
 - 看起来好像程序独占地使用主存
 - 由称为虚拟内存的内核机制提供



多进程：一种假象



■ 计算机同时运行多个进程

- 一个或多个用户的应用程序
 - 网页浏览器、电子邮件客户端、编辑器...
- 后台任务
 - 监控网络和 I/O 设备

多进程示例

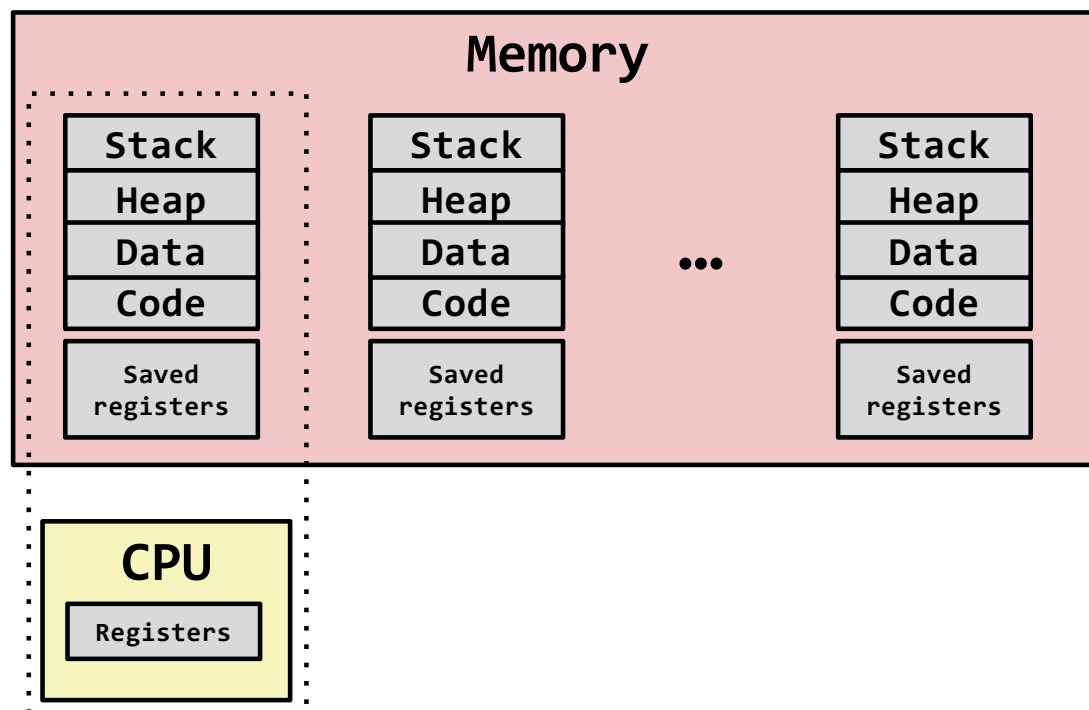
```
Processes: 123 total, 5 running, 9 stuck, 109 sleeping, 611 threads
Load Avg: 1.03, 1.13, 1.14 CPU usage: 3.27% user, 5.15% sys, 91.56% idle
SharedLibs: 576K resident, 0B data, 0B linkedit.
MemRegions: 27958 total, 1127M resident, 35M private, 494M shared.
PhysMem: 1039M wired, 1974M active, 1062M inactive, 4076M used, 18M free.
VM: 280G vsize, 1091M framework vsize, 23075213(1) pageins, 5843367(0) pageouts.
Networks: packets: 41046228/11G in, 66083096/77G out.
Disks: 17874391/349G read, 12847373/594G written.

PID    COMMAND      %CPU TIME    #TH  #WQ  #PORT #MREG RPRVT  RSHRD  RSIZE  VPRVT  VSIZE
99217-  Microsoft Of 0.0 02:28.34 4    1    202  418   21M   24M   66M   763M
99051   usbmuxd      0.0 00:04.10 3    1    47    66   436K  216K  480K  60M   2422M
99006   iTunesHelper 0.0 00:01.23 2    1    55    78   728K  3124K 1124K 43M   2429M
84286   bash         0.0 00:00.11 1    0    20    24   224K  732K  484K  17M   2378M
84285   xterm        0.0 00:00.83 1    0    32    73   656K  872K  692K  9728K 2382M
55939-  Microsoft Ex 0.3 21:58.97 10   3    360  954   16M   65M   46M   114M  1057M
54751   sleep        0.0 00:00.00 1    0    17    20   92K   212K  360K  9632K 2370M
54739   launchdadd   0.0 00:00.00 2    1    33    50   488K  220K  1736K 48M   2409M
54737   top          6.5 00:02.53 1/1  0    30    29  1416K 216K  2124K 17M   2378M
54719   automountd   0.0 00:00.02 7    1    53    64   860K  216K  2184K 53M   2413M
54701   ocspd        0.0 00:00.05 4    1    61    54  1268K 2644K 3132K 50M   2426M
54661   Grab         0.6 00:02.75 6    3   222+ 389+  15M+  26M+  40M+  75M+  2556M+
54659   cookied      0.0 00:00.15 2    1    40    61  3316K 224K  4088K 42M   2411M
53818   mdworker     0.0 00:01.67 4    1    52    91  7628K 7412K  16M   48M   2438M
50978   mdworker     0.0 00:01.51 3    1    53    91  2464K 6148K 9976K 44M   2434M
50778   emacs        0.0 00:06.70 1    0    20    35   52K   216K  88K   18M   2392M
```

■ 在 Mac 上运行程序 “top”

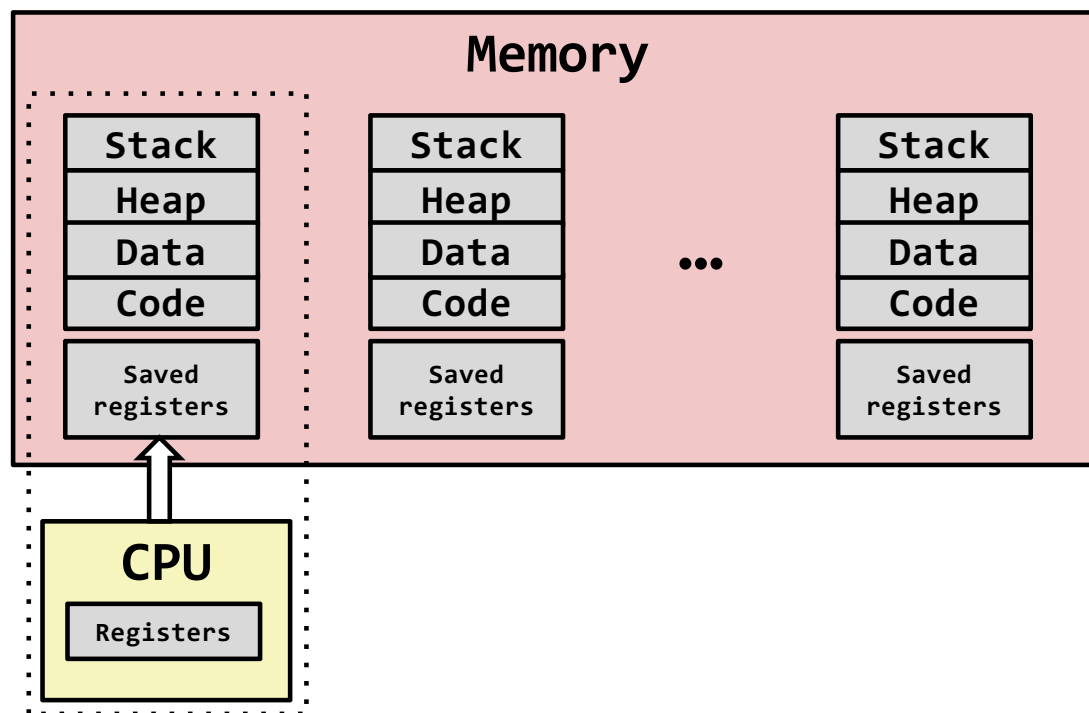
- 系统有 123 个进程，其中 5 个处于活动状态
- 由进程 ID（PID）标识

多进程：单处理器



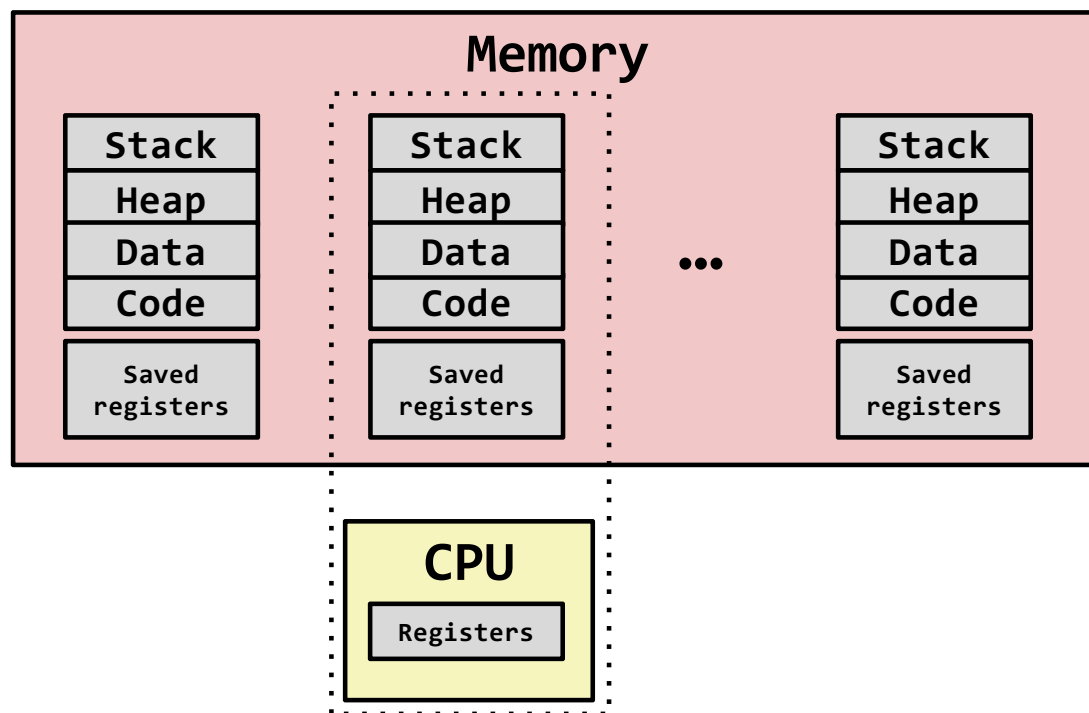
- 单个处理器同时执行多个进程
 - 进程交错执行（多任务处理）
 - 地址空间由虚拟内存系统管理
 - 非执行进程的寄存器值保存在内存中

多进程：单处理器



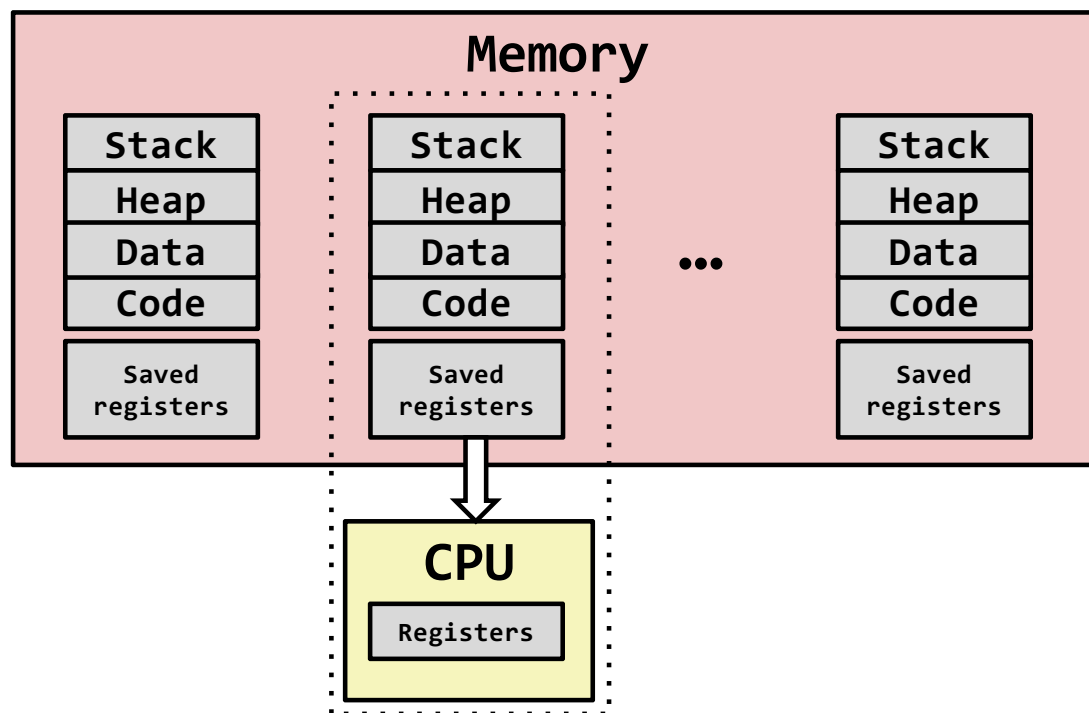
- 将当前寄存器保存到内存中

多进程：单处理器



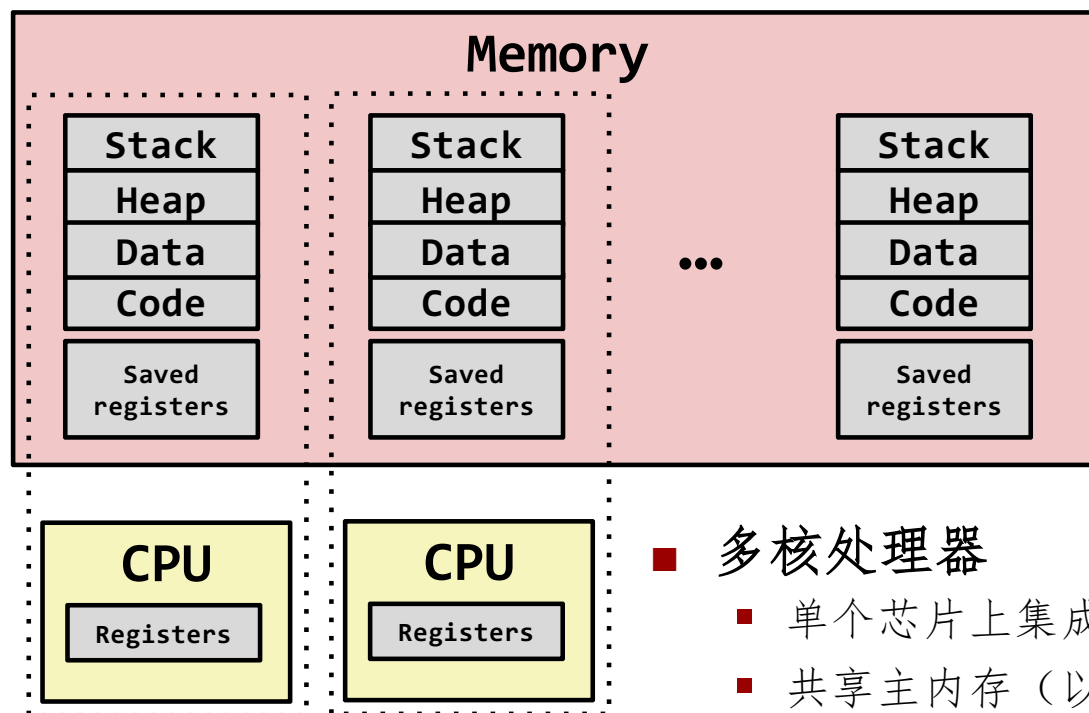
- 调度下一个进程执行

多进程：单处理器



- 加载保存的寄存器并切换地址空间（上下文切换）

多进程：多核

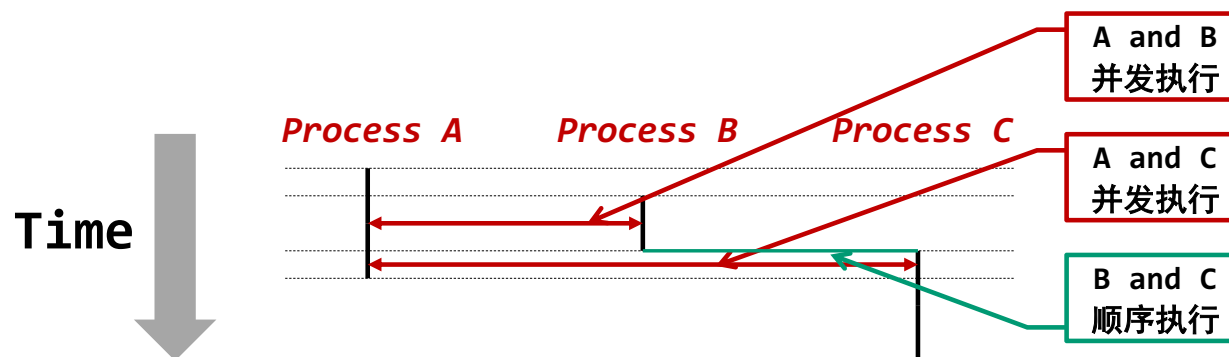


■ 多核处理器

- 单个芯片上集成多个 CPU
- 共享主内存（以及部分缓存）
- 每个核心可以执行独立的进程
 - 内核负责将进程调度到核心上执行

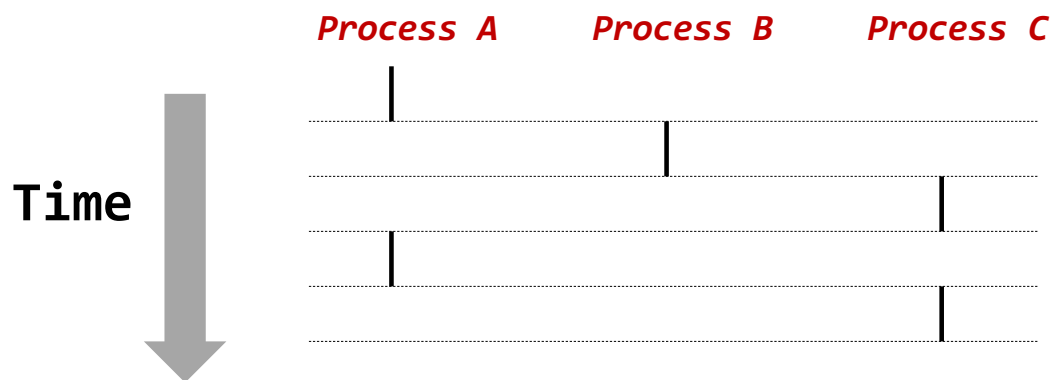
用户视角下的并发进程

- 如果两个进程的执行在时间上有重叠，它们就是**并发的**
- 否则，它们就是**顺序的**
- 并发进程看起来像是彼此并行运行
 - 这意味着它们可能会相互干扰



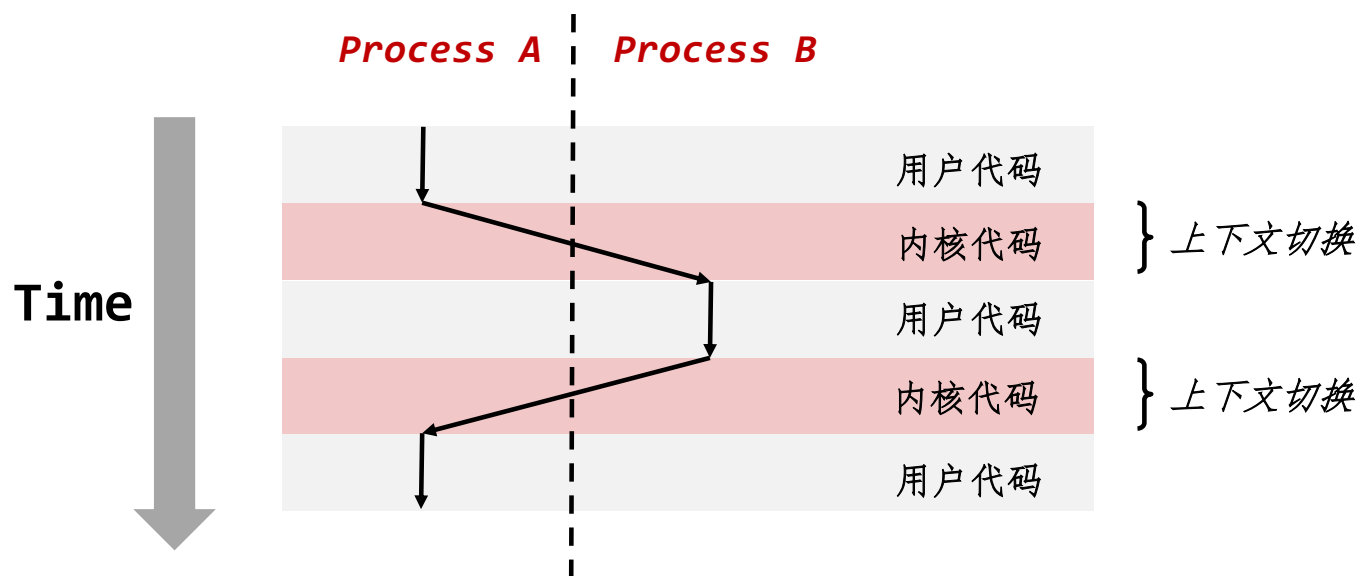
并发进程（单处理器）

- 同一时刻只有一个进程在运行
- 进程 A 和 B 的执行是交错的，并非真正的并发
- 进程 A 和 C 也是类似的情况
- 仍然可能发生进程 A 与 B 或进程 A 与 C 之间的相互干扰



上下文切换

- 进程由一个常驻内存的共享的操作系统代码块（称作**内核**）管理
 - 重要： 内核不是一个独立的进程，而是作为某个现有进程的一部分运行。
- 控制流通过**上下文切换**从一个进程转移到另一个进程



主要内容

- 异常控制流
- 异常
- 进程
- 系统调用
- 进程控制

系统调用

- 当一个程序希望产生超出自身进程范围的影响时，必须请求内核的帮助
- 例如：
 - 读取/写入文件
 - 获取当前时间
 - 分配内存（sbrk）
 - 创建新进程

```
// fopen.c
FILE *fopen(const char *fname,
            const char *mode) {
    int flags = mode2flags(mode);
    if (!flags) return NULL;
    int fd = open(fname, flags, DEFPERMS);
    if (fd == -1) return NULL;
    return fdopen(fd, mode);
}

// open.S
.global open
open:
    mov $SYS_open, %eax
    syscall
    cmp $SYS_error_thresh, %rax
    ja __syscall_error
    ret
```

系统调用的错误处理

- 出错时，Linux 系统级函数通常返回 **-1**，并设置全局变量 **errno** 以指示错误原因
- 硬性规则：
 - 必须检查每个系统级函数的返回状态
 - 唯一的例外是少数返回 `void` 的函数
- 示例：

```
if ((pid = fork()) < 0) {  
    fprintf(stderr, "fork error: %s\n", strerror(errno));  
    exit(0);  
}
```

错误报告函数

- 可以使用 错误报告函数 进行一定程度的简化

```
void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(1);
}
```

```
if ((pid = fork()) < 0)
    unix_error("fork error");
```

**Note: csapp.c exits with 0.
--> 1.**

- 出现问题时退出并不总是合适的

错误处理包装

- 使用 **Stevens** 风格的错误处理包装函数进一步简化代码

```
pid_t Fork(void)
{
    pid_t pid;

    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}
```

```
pid = Fork(); // Only returns if successful
```

- 这通常不是你在实际应用程序中希望采用的方法

主要内容

- 异常控制流
- 异常
- 进程
- 系统调用
- 进程控制

获取进程 ID

- **pid_t getpid(void)**
 - 返回当前进程的 PID
- **pid_t getppid(void)**
 - 返回父进程的 PID

创建和终止进程

从程序员的角度，我们可以认为进程总是处于下面四种状态之一：

■ Running (R)

- 进程正在执行指令，或者在有足够的 CPU 核心时 可以执行指令。

■ Blocked / Sleeping (S)

- 进程无法继续执行指令，直到某些外部事件发生（通常是 I/O 事件）

■ Stopped (T)

- 进程被用户操作（如按下 Control-Z）阻止执行。

■ Terminated / Zombie (Z)

- 进程已经完成，但其父进程尚未收到通知。

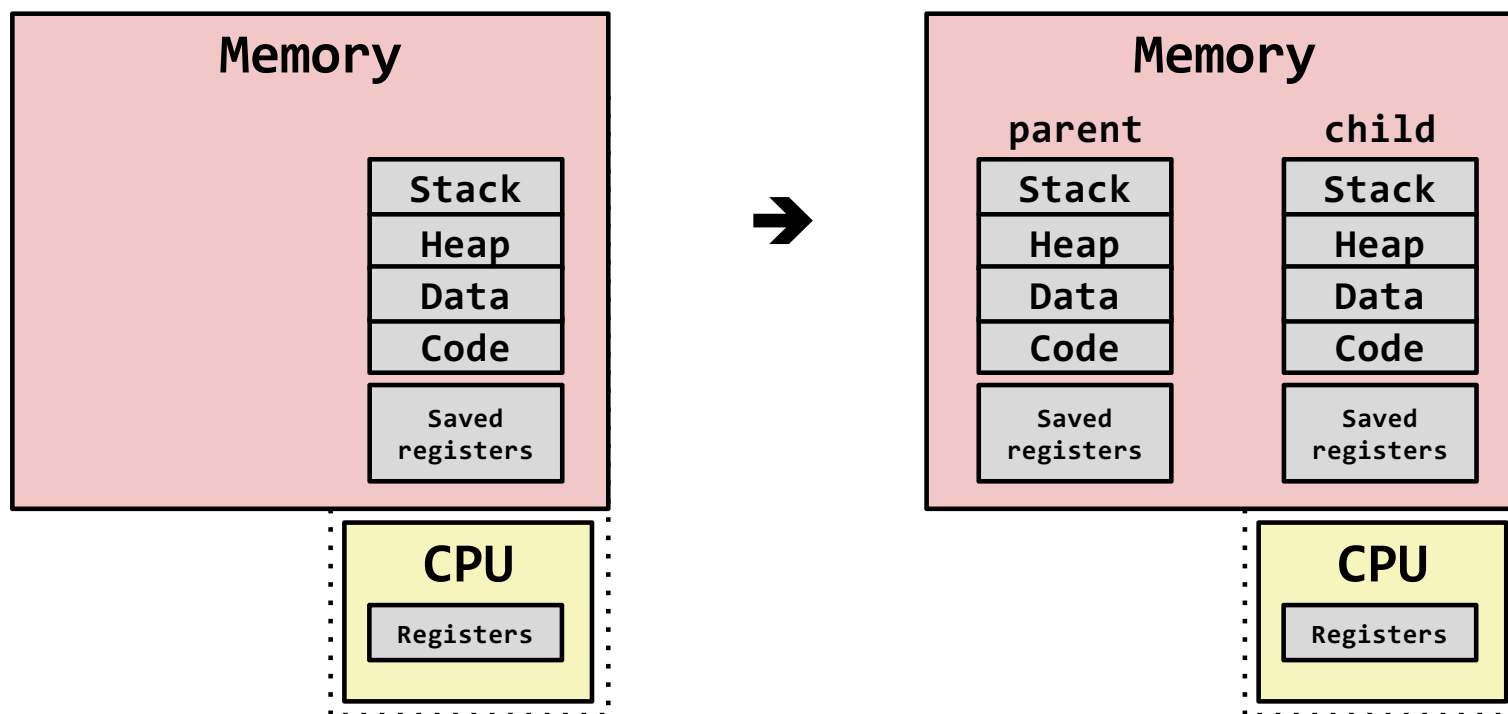
终止进程

- 进程会因为三种原因终止：
 - 收到一个信号，其默认行为是终止进程（详见下一节）
 - 从 `main` 程序返回
 - 调用 `exit` 函数
- **`void exit(int status)`**
 - 以 `status` 退出状态来终止进程
 - 约定：正常返回状态为 `0`，错误时为非零
 - 另一种设置退出状态的方法是从 `main` 程序中返回一个整数值
- **`exit` 被调用一次但从~~不~~返回**

创建进程

- 父进程通过调用 **fork** 函数创建一个新的运行的子进程
- **int fork(void)**
 - 返回 0 给子进程，返回子进程的 PID 给父进程
 - 子进程几乎和父进程一模一样
 - 子进程获得与父进程用户级虚拟地址空间相同的（但是独立的）一份副本
 - 子进程获得与父进程打开文件描述符相同的一份副本
 - 子进程的 PID 与父进程不同
- **fork** 函数是有趣的（也常常令人迷惑），因为它被调用一次，却会返回两次

fork 的概念视图



- 创建执行状态的完整副本：
 - 指定一个为父进程，一个为子进程
 - 恢复父进程或子进程的执行
 - （优化：使用写时复制（`copy-on-write`）避免内存拷贝）

fork 示例

```
int main(int argc, char** argv)
{
    pid_t pid;
    int x = 1;

    pid = Fork();
    if (pid == 0) { /* Child */
        printf("child : x=%d\n", ++x);
        return 0;
    }

    /* Parent */
    printf("parent: x=%d\n", --x);
    return 0;
}                                     fork.c
```

```
linux> ./fork
parent: x=0
child : x=2
```

- 调用一次，返回两次
- 并发执行
 - 无法预测父子进程的执行顺序
- 两份同样但独立的地址空间
 - 当 **fork** 返回时，**x** 在父进程和子进程中都为 1
 - 对 **x** 的后续更改是独立的
- 共享文件
 - **stdout** 在父进程和子进程中都是一样的

fork示例2

```
int main(int argc, char** argv)
{
    pid_t pid;
    int x = 1;

    pid = Fork();
    if (pid == 0) { /* Child */
        printf("\t>> child sleeps for 1s...\n");
        sleep(1);
        printf("\t>> child : x=%d\n", ++x);
        return 0;
    }

    /* Parent */
    printf("parent sleeps for 2s...\n");
    sleep(2);
    printf("parent: x=%d\n", --x);
    return 0;
}
```

- sleep(n) 函数会让进程进入sleeping状态，并持续n秒
- Fork从内核返回时，
 - 首先恢复parent进程，调用第一个printf
 - 然后parent进程主动进入sleeping状态2秒
 - 这时，CPU会调度child进程运行，并调用第一个printf
 - 接下来，child进程sleep 1秒，继续调用第2个printf；并结束child进程
 - CPU调度parent进程运行，继续调用第2个printf，并结束parent进程

```
linux> ./fork
parent sleeping for 2s...
        >> child sleeping for 1s...
        >> child : x=2
parent: x=0
```

fork示例3

```
int main(int argc, char** argv)
{
    pid_t pid;
    int x = 1;

    pid = Fork();
    if (pid == 0) { /* Child */
        printf("\t>> child sleeps for 2s...\n");
        sleep(2);
        printf("\t>> child : x=%d\n", ++x);
        return 0;
    }

    /* Parent */
    printf("parent sleeps for 2s...\n");
    sleep(2);
    printf("parent: x=%d\n", --x);
    return 0;
}
```

- sleep(n) 函数会让进程进入sleeping状态，并持续n秒
- Fork从内核返回时，
 - 首先恢复父进程，调用第一个printf，随后父进程主动进入sleeping状态2秒
 - 这时，CPU会调度子进程运行，并调用第一个printf，随后子进程sleep 2秒
 - 这时，CPU会调度父进程运行，并调用第2个printf，并结束父进程
 - 随后，CPU会调度子进程，并结束子进程

```
linux> ./fork
parent sleeping for 2s...
        >> child sleeping for 2s...
parent: x=0
        >> child : x=2
```

gdb调试多进程

```
linux> gcc -g fork.c -o fork
linux> gdb ./fork
(gdb) set follow-fork-mode child
(gdb) b 28 # 在if(pid == 0)处设置断点
(gdb) run # 进程运行到断点
```

...接下来可以单步跟踪到child进程

- gdb默认调试父进程
- 选项控制调试子进程
 - set follow-fork-mode child
 - 调试

创建不同的进程（直接替换自身）

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

int main(int argc, char** argv)
{
    pid_t pid;
    int x = 1;

    // 要执行的程序路径
    char *program = "./hello";
    // 参数列表，最后一个元素必须是 NULL
    char *args[] = {NULL};

    // 使用 execv 替换当前进程映像
    int ret = execv(program, args);
    printf("Never running\n");
    return 0;
}
```

execv.c

- `int execv(char *prog, char **args)`
 - 将**prog**路径文件替换当前进程的内存映像
 - 使用**args**参数，运行当前进程

```
linux> ./exec
>> hello sleeps for 2s...
>> hello...
```

创建不同的进程（创建子进程）

```
int main(int argc, char** argv)
{
    pid_t pid;
    int x = 1;

    pid = Fork();
    if (pid == 0) { /* Child */
        // 要执行的程序路径
        char *program = "./hello";
        // 参数列表, 最后一个元素必须是 NULL
        char *args[] = {NULL};
        // 使用 execv 替换当前进程映像
        int ret = execv(program, args);
    }

    /* Parent */
    printf("parent sleeps for 2s...\n");
    sleep(2);
    printf("parent: x=%d\n", --x);
    return 0;
}
```

exec1.c

```
linux> ./exec1
parent sleeps for 2s...
        >> hello sleeps for 2s...
parent: x=0
        >> hello...
```

使用进程图对 **fork** 进行建模

- **进程图**是刻画并发程序中语句的偏序关系的工具
 - 每个顶点对应于一条语句的执行
 - 有向边 $a \rightarrow b$ 表示语句 a 发生在语句 b 前
 - 边上可以标记变量的当前值
 - 对应于 `printf` 的顶点可以标记输出
 - 每个图以没有入边的顶点开始
- **进程图**的任何**拓扑排序**都对应一个可行的全序排列
 - 顶点的全序排列中所有边都是从左向右的

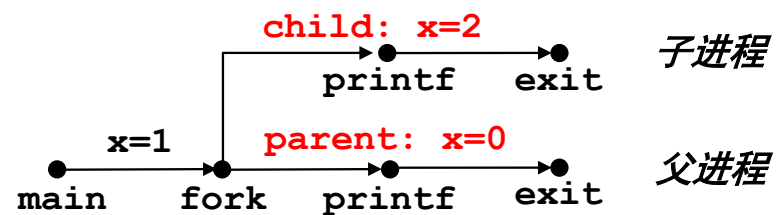
进程图实例

```
int main()
{
    pid_t pid;
    int x = 1;

    pid = Fork();
    if (pid == 0) { /* Child */
        printf("child : x=%d\n", ++x);
        exit(0);
    }

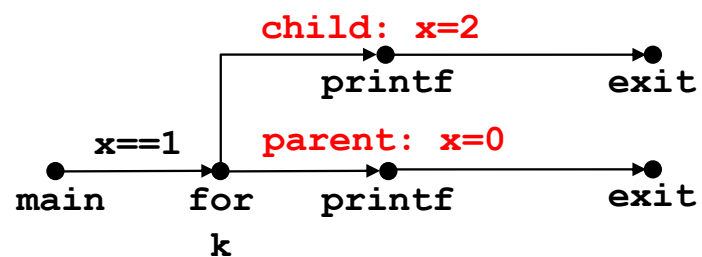
    /* Parent */
    printf("parent: x=%d\n", --x);
    exit(0);
}
```

fork.c

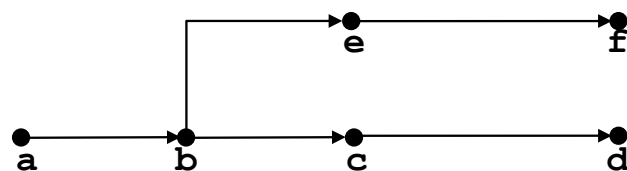


解读进程图

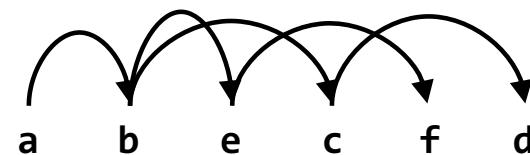
■ 原始图:



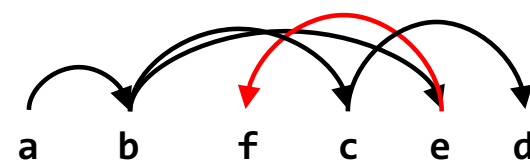
■ 重新标号的进程图:



可行的全序排序:



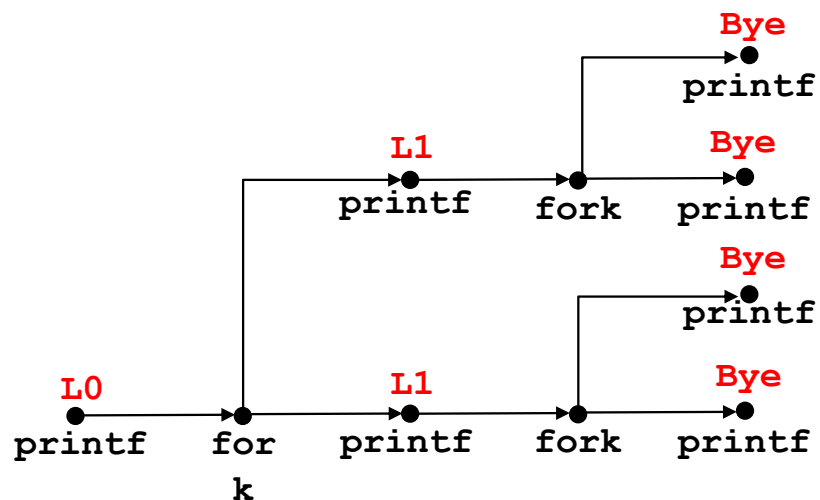
不可行的全序排序:



fork 示例：两个连续的 fork

```
void fork2()
{
    printf("L0\n");
    fork();
    printf("L1\n");
    fork();
    printf("Bye\n");
}
```

forks.c



可行的输出：

L0
L1
Bye
Bye
L1
Bye
Bye

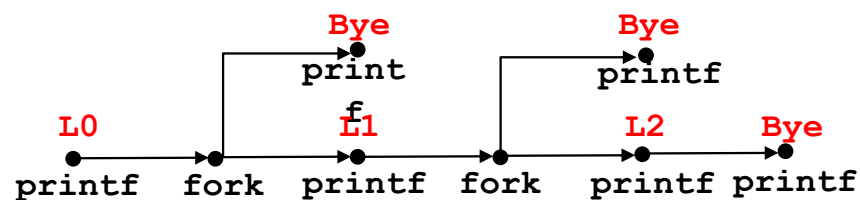
不可行的输出：

L0
Bye
L1
Bye
L1
Bye
Bye

fork 示例：父进程中的嵌套 fork

```
void fork4()
{
    printf("L0\n");
    if (fork() != 0) {
        printf("L1\n");
        if (fork() != 0) {
            printf("L2\n");
        }
    }
    printf("Bye\n");
}
```

forks.c



可行的输出：

L0
L1
Bye
Bye
L2
Bye

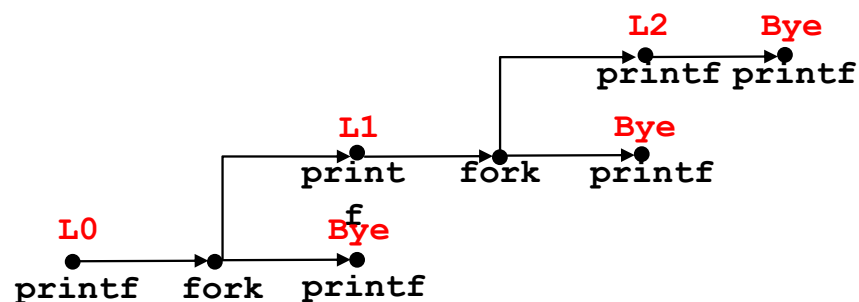
不可行的输出：

L0
Bye
L1
Bye
Bye
L2

fork 示例：子进程中的嵌套 fork

```
void fork5()
{
    printf("L0\n");
    if (fork() == 0) {
        printf("L1\n");
        if (fork() == 0) {
            printf("L2\n");
        }
    }
    printf("Bye\n");
}
```

forks.c



可行的输出：

L0
Bye
L1
L2
Bye
Bye

不可行的输出：

L0
Bye
L1
Bye
Bye
L2

回收子进程

■ 概念

- 当进程终止时，它仍然占用系统资源
 - 例如：退出状态、各种操作系统表
- 称为 **zombie**

■ 回收

- 父进程对终止的子进程执行（使用 `wait` 或 `waitpid`）
- 父进程会获得子进程的退出状态信息
- 内核随后删除僵尸子进程

■ 如果父进程未回收子进程？

- 如果父进程终止而未回收子进程，孤儿进程会被 **init** 进程回收(`pid == 1`)
- 因此，仅在长时间运行的进程中需要显式回收
 - 例如 **shell** 和服务端



Zombie 示例

```
void fork7() {  
    if (fork() == 0) {  
        /* Child */  
        printf("Terminating Child, PID = %d\n", getpid());  
        exit(0);  
    } else {  
        printf("Running Parent, PID = %d\n", getpid());  
        while (1)  
            ; /* Infinite loop */  
    }  
}
```

forks.c

```
linux> ./forks 7 &  
[1] 6639  
Running Parent, PID = 6639  
Terminating Child, PID = 6640  
linux> ps  
  PID TTY          TIME CMD  
 6585 tttyp9      00:00:00 tcsh  
 6639 tttyp9      00:00:03 forks  
 6640 tttyp9      00:00:00 forks <defunct>  
 6641 tttyp9      00:00:00 ps  
linux> kill 6639  
[1] Terminated  
linux> ps  
  PID TTY          TIME CMD  
 6585 tttyp9      00:00:00 tcsh  
 6642 tttyp9      00:00:00 ps
```

ps 命令显示子进程为 “defunct” (即 zombie)

终止父进程使得子进程被 **init** 回收

非终止子进程示例

```
linux> ./forks 8
Terminating Parent, PID = 6675
Running Child, PID = 6676
linux> ps
  PID TTY          TIME CMD
 6585 tttyp9      00:00:00 tcsh
 6676 tttyp9      00:00:06 forks
 6677 tttyp9      00:00:00 ps
linux> kill 6676
linux> ps
  PID TTY          TIME CMD
 6585 tttyp9      00:00:00 tcsh
 6678 tttyp9      00:00:00 ps
```

```
void fork8()
{
    if (fork() == 0) {
        /* Child */
        printf("Running Child, PID = %d\n", getpid());
        while (1)
            ; /* Infinite loop */
    } else {
        printf("Terminating Parent, PID = %d\n", getpid());
        exit(0);
    }
}
```

forks.c

子进程仍然活动，即使父进程已经终止

必须显式终止子进程，否则它将无限期运行

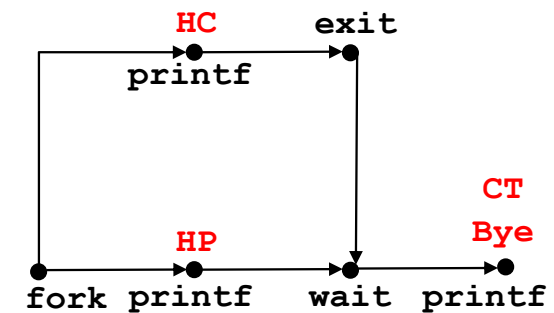
wait: 与子进程同步

- 父进程通过这些系统调用之一回收子进程:
- **pid_t wait(int *status)**
 - 挂起当前进程，直到其一个子进程终止
 - 返回子进程的 PID，并在 **status** 中记录退出状态
- **pid_t waitpid(pid_t pid, int *status, int options)**
 - **wait** 的更灵活版本:
 - 可以等待特定的子进程或子进程组
 - **WNOHANG**: 如果没有子进程要回收，可以立即返回

wait: 与子进程同步

```
void fork9() {  
    int child_status;  
  
    if (fork() == 0) {  
        printf("HC: hello from child\n");  
        exit(0);  
    } else {  
        printf("HP: hello from parent\n");  
        wait(&child_status);  
        printf("CT: child has terminated\n");  
    }  
    printf("Bye\n");  
}
```

forks.c



可行的输出:

HC
HP
CT
Bye

不可行的输出:

HP
CT
Bye
HC

wait: 状态代码

- **wait** 的返回值是终止的子进程的 **PID**
- 如果 **status != NULL**, 那么它指向的整数将被设置为指示退出状态的值
 - 比传递给 **exit** 的值更多的信息
 - 必须使用 **sys/wait.h** 中定义的宏来解读
 - **WIFEXITED**, **WEXITSTATUS**, **WIFSIGNALED**, **WTERMSIG**,
WIFSTOPPED, **WSTOPSIG**, **WIFCONTINUED**
 - 详见教科书

另一个 `wait` 示例

- 如果多个子进程完成，将以任意顺序处理它们
- 可以使用宏 `WIFEXITED` 和 `WEXITSTATUS` 获取有关退出状态的信息

```
void fork10() {
    pid_t pid[N];
    int i, child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0) {
            exit(100+i); /* Child */
        }
    for (i = 0; i < N; i++) { /* Parent */
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                  wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

forks.c

waitpid: 等待特定进程

- `pid_t waitpid(pid_t pid, int &status, int options)`
 - 挂起当前进程，直到特定进程终止
 - 各种选项（见教科书）

```
void fork11() {
    pid_t pid[N];
    int i;
    int child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            exit(100+i); /* Child */
    for (i = N-1; i >= 0; i--) {
        pid_t wpid = waitpid(pid[i], &child_status, 0);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                    wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

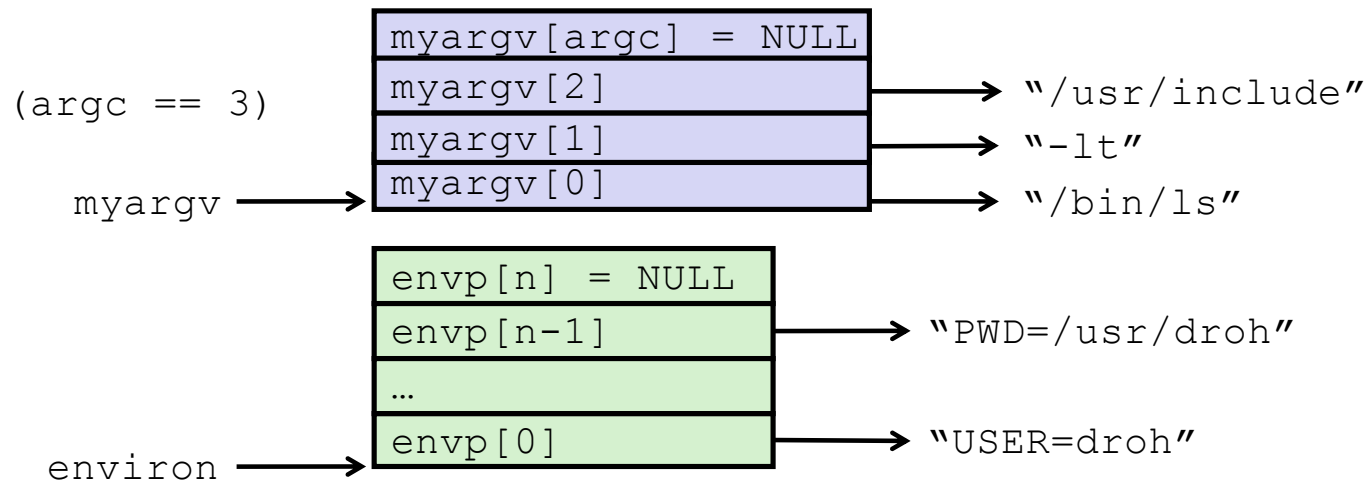
forks.c

execve: 加载和运行程序

- `int execve(char *filename, char *argv[], char *envp[])`
- 在当前进程中加载和运行:
 - 可执行文件 `filename`
 - 可以是目标文件或以 `#!/interpreter` 开头的脚本文件 (例如 `#!/bin/bash`)
 - 参数列表 `argv`
 - 按约定, `argv[0]==filename`
 - 环境变量列表 `envp`
 - 格式为 “name=value” 的字符串 (例如 `USER=droh`)
 - `getenv`, `putenv`, `printenv`
- 覆盖代码段、数据段和栈
 - 保留 `PID`、打开的文件和信号上下文
- 调用一次, 永不返回
 - 除非有错误

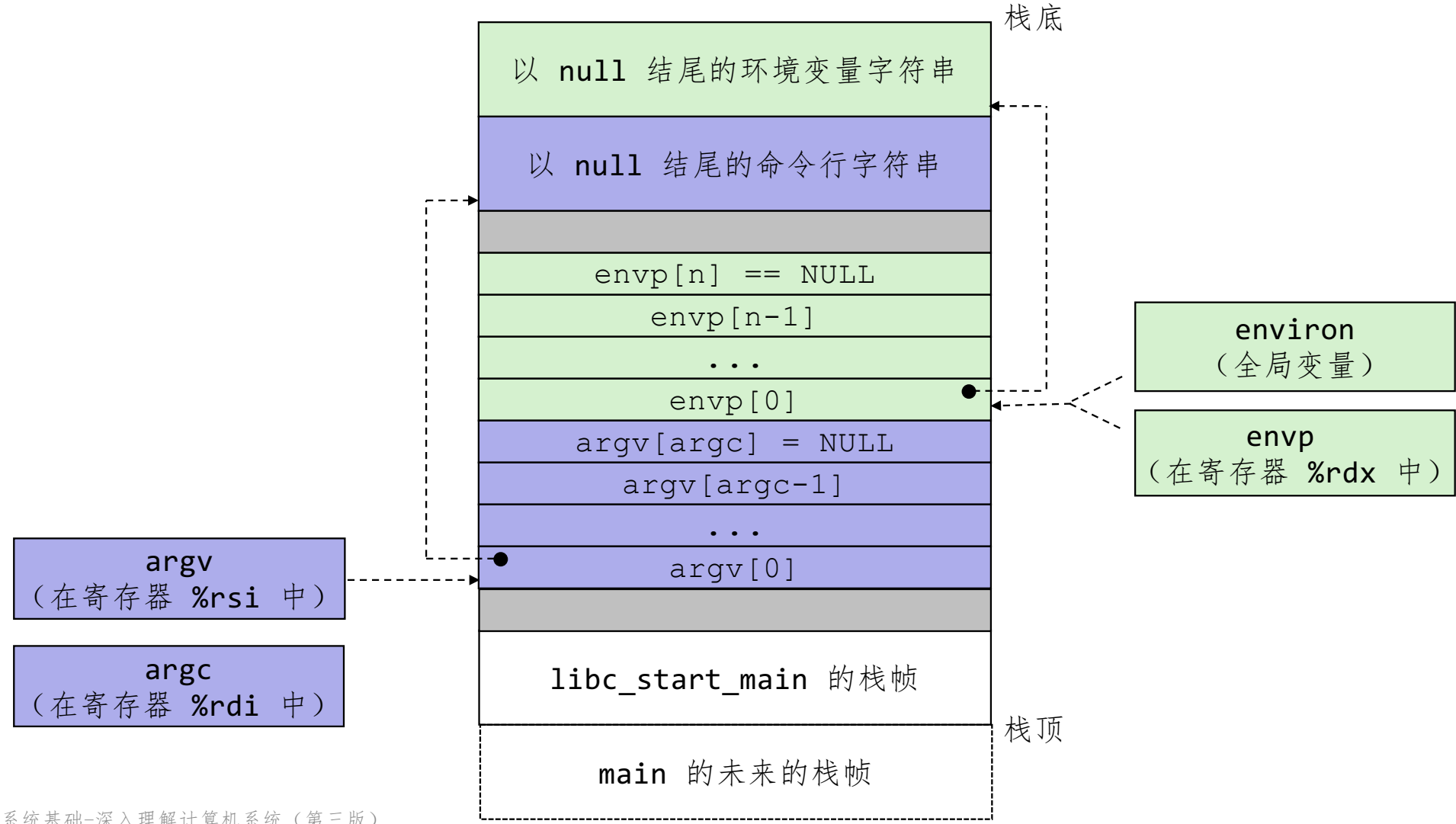
execve 示例

- 使用当前环境在子进程中执行 `"/bin/ls -lt /usr/include"` :

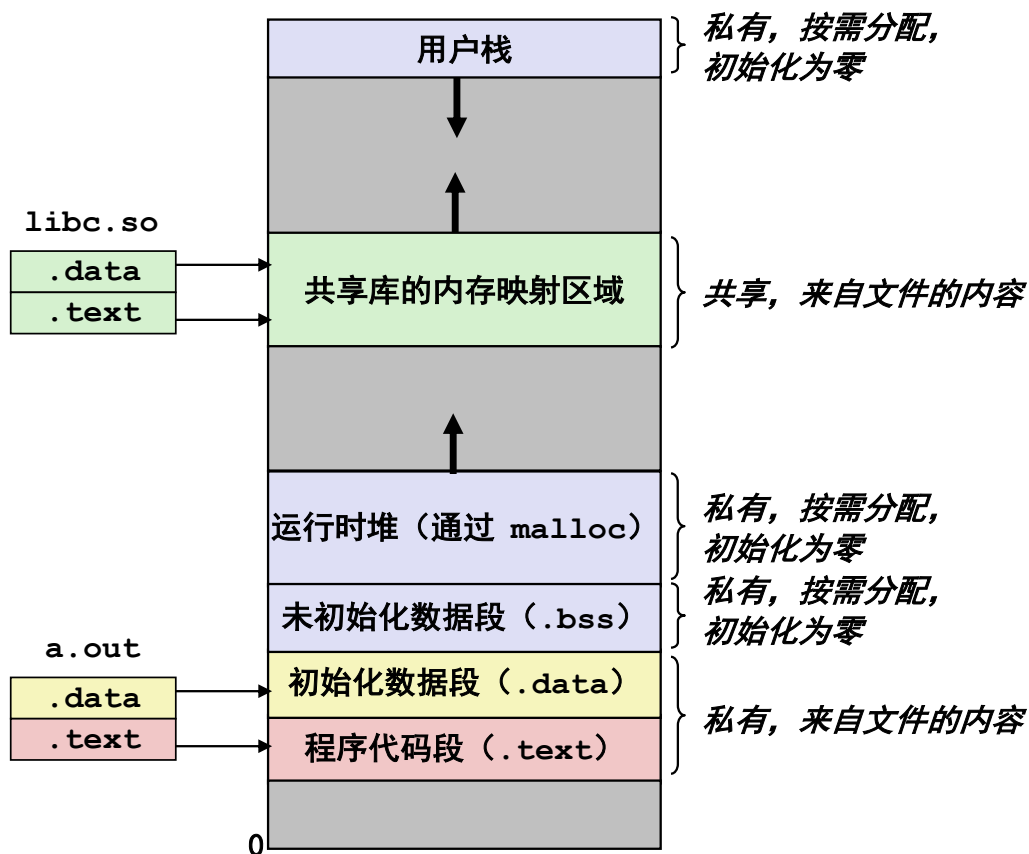


```
if ((pid = Fork()) == 0) { /* Child runs program */
    if (execve(myargv[0], myargv, environ) < 0) {
        printf("%s: Command not found.\n", myargv[0]);
        exit(1);
    }
}
```

新程序启动时的栈结构



execve 和进程内存布局



- 在当前进程中通过 `execve` 加载并运行新程序 `a.out` :
- 释放旧区域的 `vm_area_struct` 和页表
- 为新区域创建 `vm_area_struct` 和页表
 - 程序和已初始化数据由目标文件支持
 - `.bss` 和栈由匿名文件支持
- 设置 `PC` 为 `.text` 段的入口点
 - Linux 将根据需要访问代码和数据页面时发生缺页异常

总结

■ 异常

- 需要非标准控制流的事件
- 外部生成（中断）或内部生成（陷阱和故障）

■ 进程

- 系统在任意给定时间有多个活动进程
- 但同一时间在单个核心上只能有一个进程执行
- 每个进程看起来都可以完全控制处理器和私有内存空间

总结（续）

■ 创建进程

- 调用 `fork`
- 一次调用，两次返回

■ 进程完成

- 调用 `exit`
- 一次调用，不返回

■ 回收和等待进程

- 调用 `wait` 或 `waitpid`

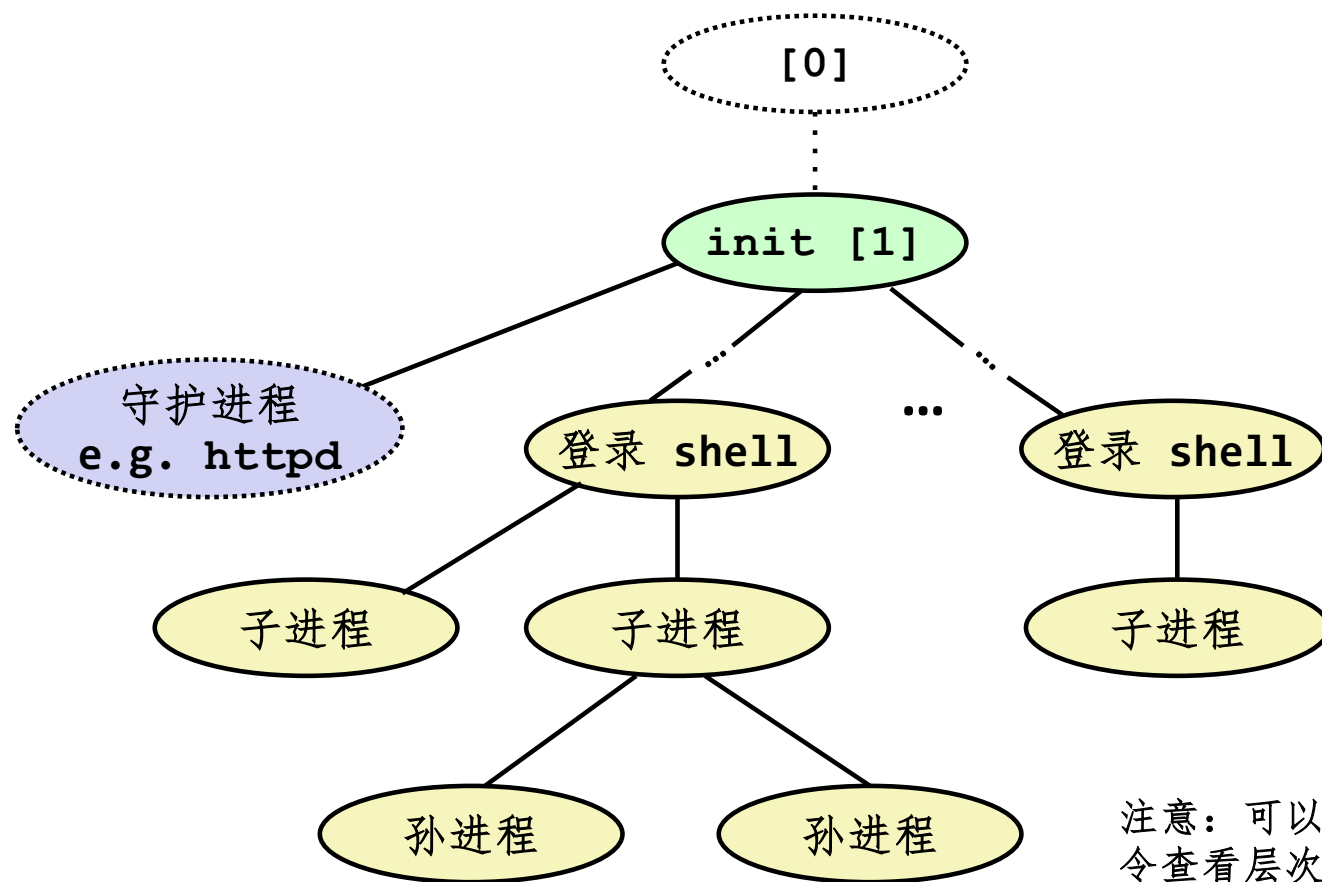
■ 加载和运行程序

- 调用 `execve`（或其变体）
- 一次调用，（通常）不返回

主要内容

- Shell
- 信号
- 非本地跳转

Linux 进程层次结构

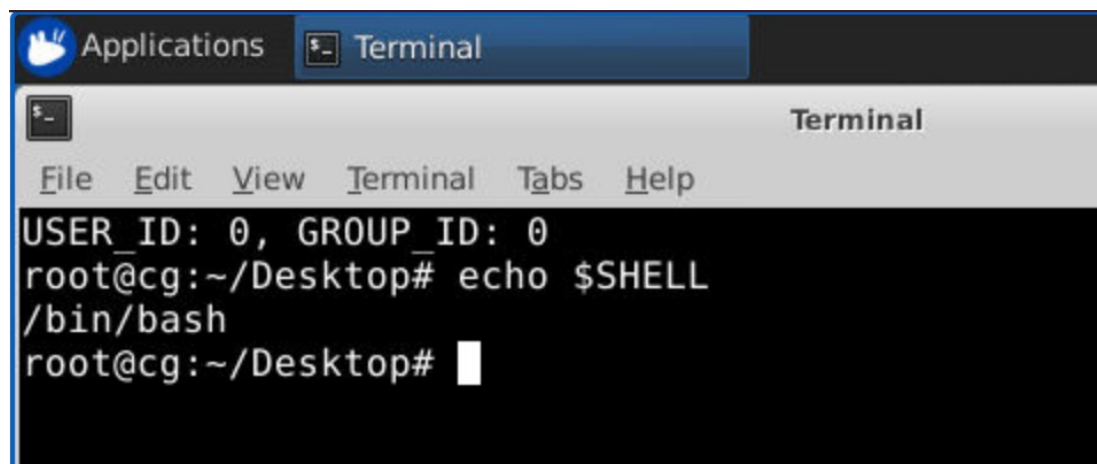


注意：可以使用 **Linux** `ps tree` 指令查看层次结构

Shell 程序

■ *shell* 是代表用户运行程序的应用程序

- **sh** Original Unix shell (Stephen Bourne, AT&T Bell Labs, 1977)
- **csh/tcsh** BSD Unix C shell
- **bash** “Bourne-Again” Shell (default Linux shell)



A screenshot of a Linux terminal window. The window title bar shows 'Applications' and 'Terminal'. The terminal content displays the following text:

```
USER_ID: 0, GROUP_ID: 0
root@cg:~/Desktop# echo $SHELL
/bin/bash
root@cg:~/Desktop#
```

Shell 程序

■ 简单的 shell

- 教科书 524 页起
- 实现一个非常基本的 shell
- 目的
 - 了解你键入命令时会发生什么
 - 理解进程控制操作的使用和运行

```
linux> ./shellex
> /bin/ls -l csapp.c
-rw-r--r-- 1 bryant users 23053 Jun 15 2015 csapp.c
> /bin/ps
  PID TTY          TIME CMD
 31542 pts/2        00:00:01 tcsh
 32017 pts/2        00:00:00 shellex
 32019 pts/2        00:00:00 ps
> /bin/sleep 10 &
32031 /bin/sleep 10 &
> /bin/ps
  PID TTY          TIME CMD
 31542 pts/2        00:00:01 tcsh
 32024 pts/2        00:00:00 emacs
 32030 pts/2        00:00:00 shellex
 32031 pts/2        00:00:00 sleep
 32033 pts/2        00:00:00 ps
> quit
```

简单的 Shell 实现

■ 基本循环

- 从命令行读取一行输入
- 执行请求的操作
 - 内置命令（目前只实现了 `quit`）
 - 从文件加载并执行程序

执行是按顺序进行的读/求值步骤

```
int main()
{
    char cmdline[MAXLINE]; /* command line */

    while (1) {
        /* read */
        printf("> ");
        Fgets(cmdline, MAXLINE, stdin);
        if (feof(stdin))
            exit(0);

        /* evaluate */
        eval(cmdline);
    }
}
```

shellex.c

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

shellex.c

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

← parseline 解析 'buf'，放到 argv 中，并返回输入行是否以 '&' 结尾

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

忽略空行

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

如果是内置命令，则在此程序中处理；否则，执行 **fork/exec** 来运行 **argv[0]** 指定的程序

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

创建子进程

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

启动 **argv[0]**
execve 仅在发生错误时返回

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

如果子进程在前台运行，则等待其完成。

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

如果子进程在后台运行，打印其 **PID** 并继续执行其他任务。

简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

糟糕！这个程序是有缺陷的。

shellex.c

示例 Shell 的缺陷

- Shell 设计的目标是持续运行

- 不应积累不必要的资源
 - 内存
 - 子进程
 - 文件描述符

- 示例 Shell 可以正确地等待并回收前台任务

- 但后台任务呢？

- 终止后会变成僵尸进程
- 因为 Shell （通常）不会终止，这些僵尸进程将永远不会被回收
- 会导致内存泄漏，可能耗尽内存

异常控制流 ECF 来拯救！

■ 解决方案：异常控制流

- 内核会中断常规处理以通知后台进程完成
- 在 **Unix** 中，这种通知机制称为 **信号**

主要内容

- Shell
- 信号
- 非本地跳转

信号

- 一个 **信号** 就是一条小消息，它通知进程系统中发生了一个某种类型的事件
 - 类似于异常和中断
 - 从内核发送（有时由其他进程请求）到目标进程
 - 信号类型通过 **ID (1-30)** 标识
 - 信号仅包含其 **ID** 和它到达的事实，没有其他信息

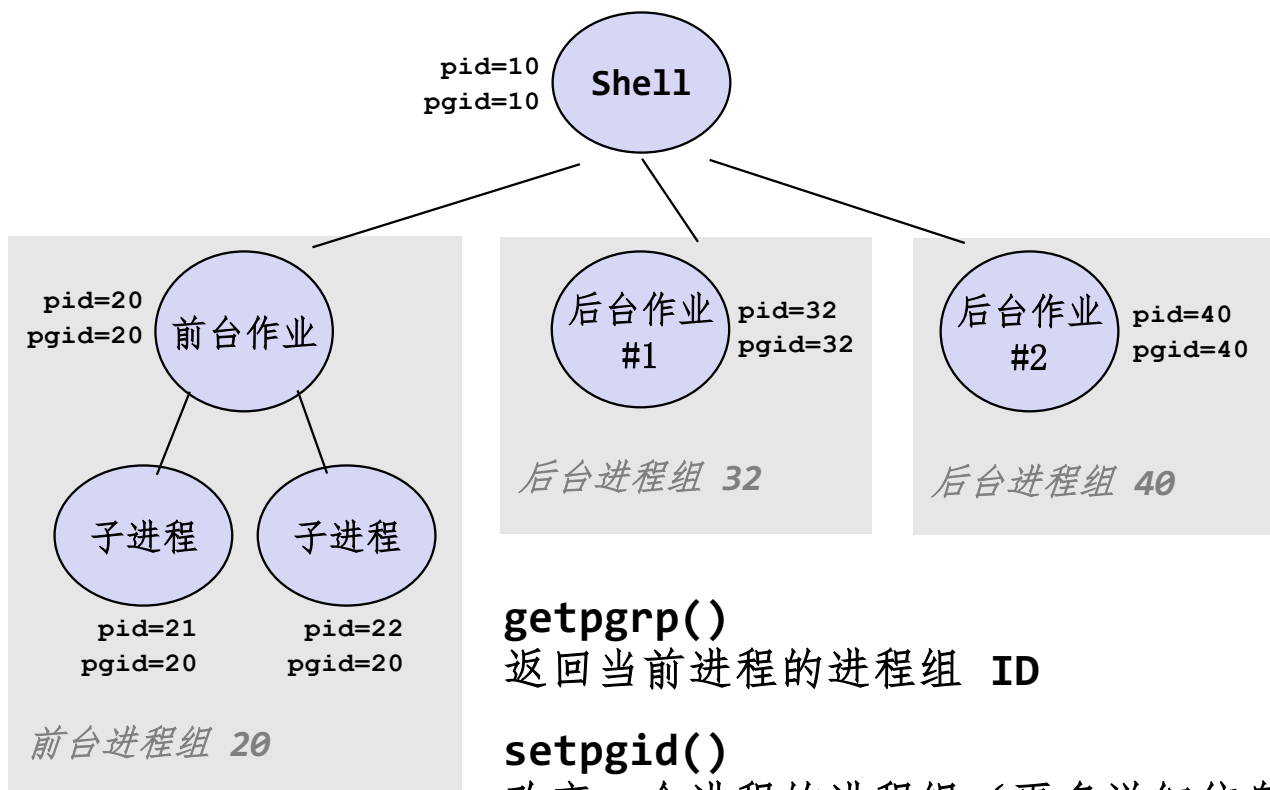
<i>ID</i>	<i>名称</i>	<i>默认行为</i>	<i>相应事件</i>
2	SIGINT	终止	用户键入 Ctrl-c
9	SIGKILL	终止	杀死程序（不能被覆盖或忽略）
11	SIGSEGV	终止	段故障
14	SIGALRM	终止	定时器信号
17	SIGCHLD	忽略	子进程停止或终止

信号术语：发送信号

- 内核通过更新目的进程上下文中的某个状态，**发送**（递送）一个信号给**目的进程**
- 内核发送信号可能有以下原因：
 - 内核检测到一个系统事件，比如除零错误（SIGFPE）或者子进程终止（SIGCHLD）
 - 一个进程调用了 **kill** 函数，显式地要求内核发送一个信号给目的进程

发送信号：进程组

- 每个进程都只属于一个进程组



getpgrp()

返回当前进程的进程组 ID

setpgid()

改变一个进程的进程组（更多详细信息请参阅教材）

用 `/bin/kill` 程序发送信号

- `/bin/kill` 可以向进程或进程组发送任意的信号

- 示例

- `/bin/kill -9 24818`

发送信号9 (SIGKILL) 给进程 24818

- `/bin/kill -9 -24817`

发送信号9 (SIGKILL) 给进程组 24817 中的每一个进程

```
linux> ./forks 16
Child1: pid=24818 pgrp=24817
Child2: pid=24819 pgrp=24817
```

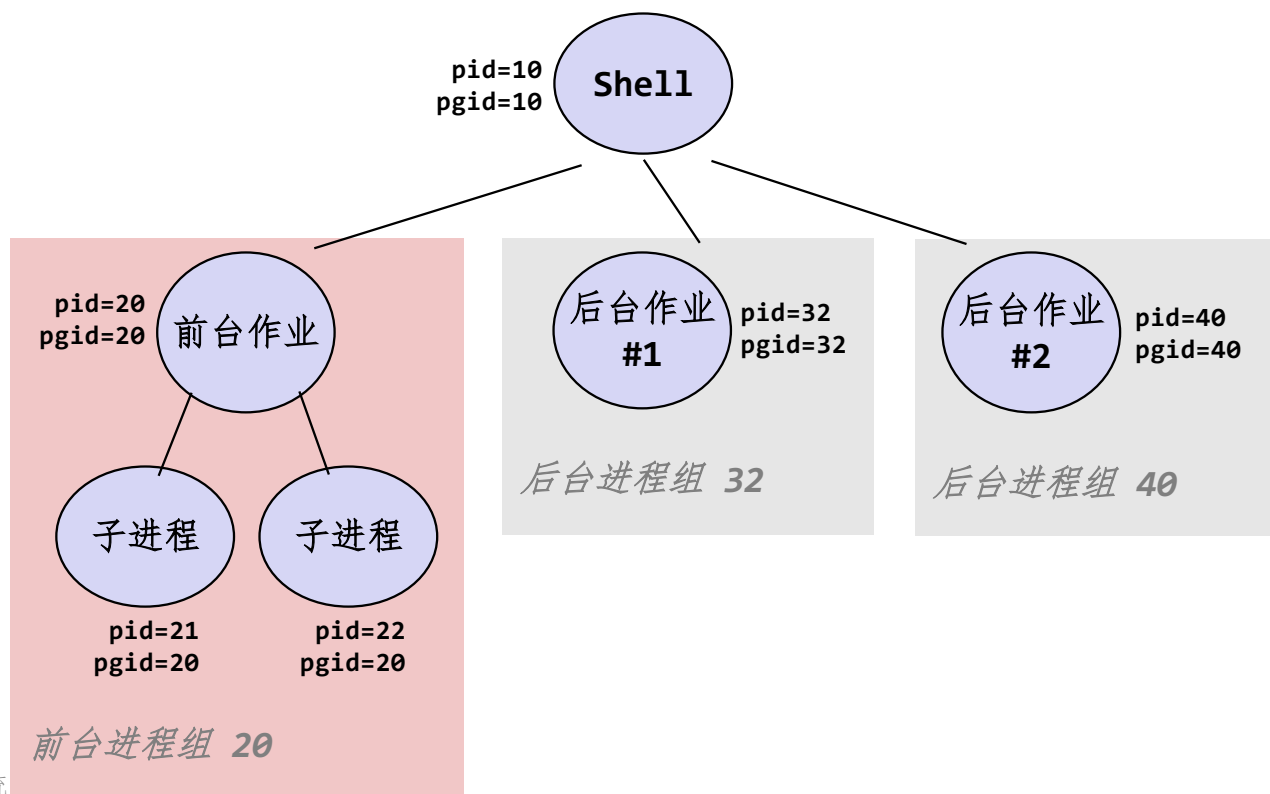
```
linux> ps
  PID TTY          TIME CMD
24788 pts/2        00:00:00 tcsh
24818 pts/2        00:00:02 forks
24819 pts/2        00:00:02 forks
24820 pts/2        00:00:00 ps
```

```
linux> /bin/kill -9 -24817
```

```
linux> ps
  PID TTY          TIME CMD
24788 pts/2        00:00:00 tcsh
24823 pts/2        00:00:00 ps
linux>
```

从键盘发送信号

- 键入 **ctrl-c** (**ctrl-z**) 会导致内核发送 **SIGINT** (**SIGTSTP**) 信号到前台进程组中的每个作业
 - **SIGINT** - 默认操作是终止每个进程
 - **SIGTSTP** - 默认操作是停止（暂停）每个进程



ctrl-c 和 ctrl-z 的例子

```
bluefish> ./forks 17
Child: pid=28108 pgrp=28107
Parent: pid=28107 pgrp=28107
<types ctrl-z>
Suspended
bluefish> ps w
  PID TTY          STAT       TIME COMMAND
 27699 pts/8        Ss          0:00   -tcsh
 28107 pts/8        T           0:01   ./forks 17
 28108 pts/8        T           0:01   ./forks 17
 28109 pts/8        R+          0:00   ps w
bluefish> fg
./forks 17
<types ctrl-c>
bluefish> ps w
  PID TTY          STAT       TIME COMMAND
 27699 pts/8        Ss          0:00   -tcsh
 28110 pts/8        R+          0:00   ps w
```

STAT（进程状态）图例：

第一个字母：

S: sleeping 睡眠

T: stopped 停止

R: running 运行中

第二个字母：

s: session leader 会话的领导进程

+: 前台进程组

更多详细信息请参阅 “man ps”

用 `kill` 函数发送信号

```
void fork12()
{
    pid_t pid[N];
    int i;
    int child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0) {
            /* Child: Infinite Loop */
            while(1)
                ;
        }

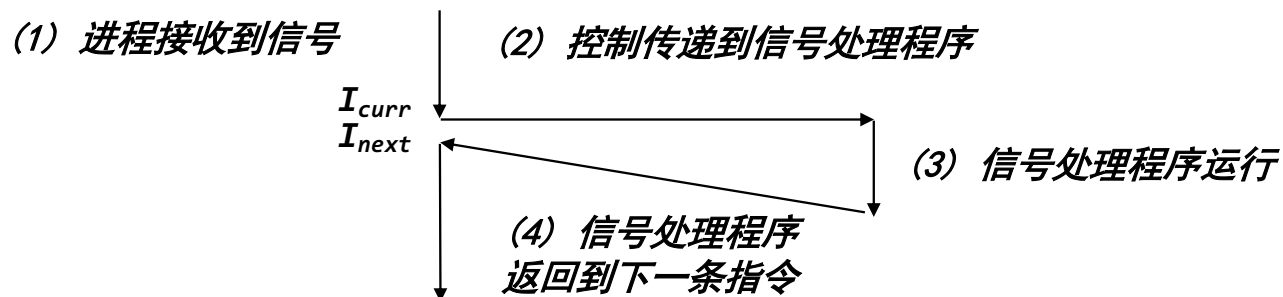
    for (i = 0; i < N; i++) {
        printf("Killing process %d\n", pid[i]);
        kill(pid[i], SIGINT);
    }

    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}
```

forks.c

信号术语：接收信号

- 当目的进程被内核强迫以某种方式对信号的发送做出反应时，它就**接收**了信号
- 可能的反应方式：
 - **忽略**信号（什么也不做）
 - **终止**进程（可能会 core dump）
 - **捕获**信号，通过执行一个称为**信号处理程序**的用户级函数
 - 类似于硬件异常处理程序响应异步中断的方式



信号术语：待处理和阻塞信号

- 一个发出而没有接收的信号叫做 **待处理** *pending* 信号
 - 一种类型至多只有一种待处理信号
 - 重要提示： **信号不会排队等待**
 - 如果一个进程有一个类型为 **k** 的待处理信号，那么任何接下来发送到这个进程的类型为 **k** 的信号都会被丢弃
- 一个进程可以 **阻塞** 接收某种信号
 - 它仍可以被发送，但是产生的待处理信号不会被接收，直到进程取消对这种信号的阻塞
- 一个待处理信号最多只能被接收一次

信号术语：Pending/Blocked 位

■ 内核维护每个进程的上下文中的 **pending** 和 **blocked** 位向量

■ **pending**: 待处理信号的集合

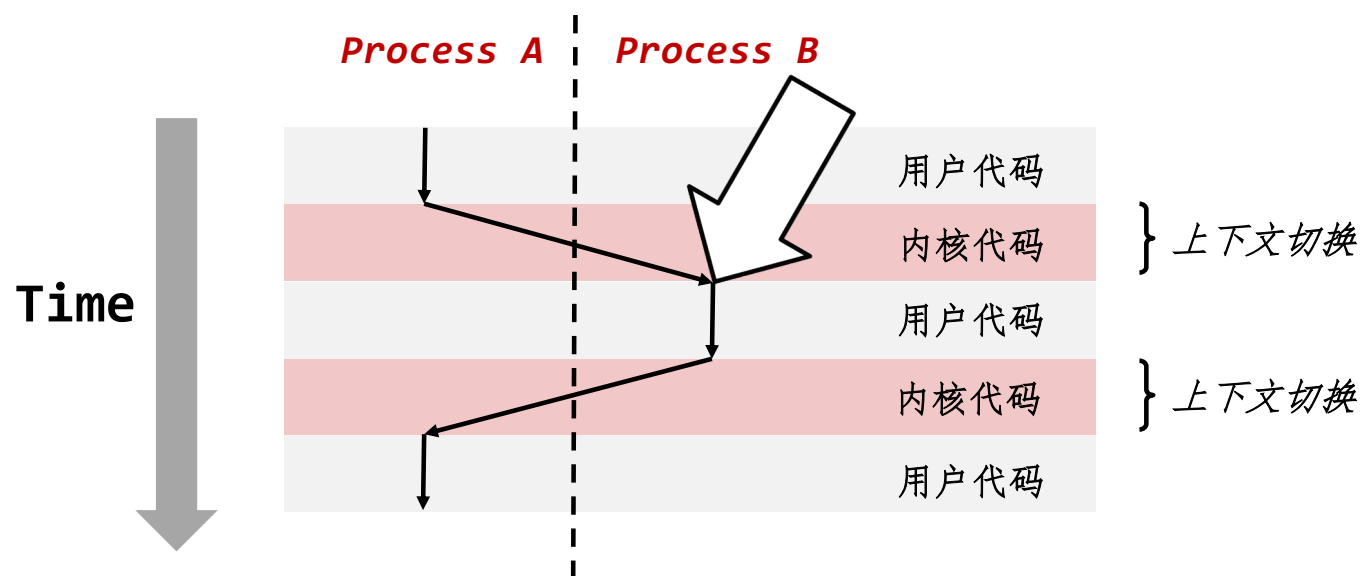
- 只要传递了一个类型为 **k** 的信号，内核就会设置 **pending** 中的第 **k** 位
- 只要接收了一个类型为 **k** 的信号，内核就会清除 **pending** 中的第 **k** 位

■ **blocked**: 阻塞信号的集合

- 可以通过 **sigprocmask** 函数设置或清除这些位
- 也称为 *信号掩码* (*signal mask*)

接收信号

- 假设内核从异常处理程序返回，并准备将控制权交给进程 p



接收信号

- 假设内核从异常处理程序返回，并准备将控制权交给进程 p
- 内核计算 $\text{pnb} = \text{pending} \& \sim\text{blocked}$
 - 进程 p 的未被阻塞的待处理信号集合
- If ($\text{pnb} == 0$)
 - 将控制权传递给 p 的逻辑控制流的下一条指令
- Else
 - 在 pnb 中选择最小非零位 k 并强制进程 p 接收信号 k
 - 接收信号会触发进程 p 的某些行为
 - 对所有 pnb 中的非零位 k 重复上述过程
 - 将控制权传递给 p 的逻辑控制流的下一条指令

默认行为

- 每个信号类型都有一个预定义的默认行为，是下面中的一种：
 - 进程终止
 - 进程停止（挂起）直到被 SIGCONT 信号重启
 - 进程忽略该信号

设置信号处理程序

- **signal** 函数用于修改与接收信号 **signum** 相关联的默认行为

- `handler_t *signal(int signum, handler_t *handler)`

- **handler** 的不同取值

- **SIG_IGN**: 忽略类型为 **signum** 的信号
- **SIG_DFL**: 类型为 **signum** 的信号行为的恢复为默认行为
- 否则, **handler** 为用户定义的 **信号处理程序** 的地址
 - 进程接收到一个类型为 **signum** 的信号时, 调用这个程序
 - 称为 **“设置”** 信号处理程序
 - 调用信号处理程序被称为 **“捕获”** 信号。执行信号处理程序被称为 **“处理”** 信号
 - 当处理程序执行它的 **return** 语句时, 控制传递回控制流中进程被信号接收中断位置处的指令

信号处理程序示例

```
void sigint_handler(int sig) /* SIGINT handler */
{
    printf("So you think you can stop the bomb with ctrl-c, do you?\n");
    sleep(2);
    printf("Well...");
    fflush(stdout);
    sleep(1);
    printf("OK. :-)\n");
    exit(0);
}

int main()
{
    /* Install the SIGINT handler */
    if (signal(SIGINT, sigint_handler) == SIG_ERR)
        unix_error("signal error");

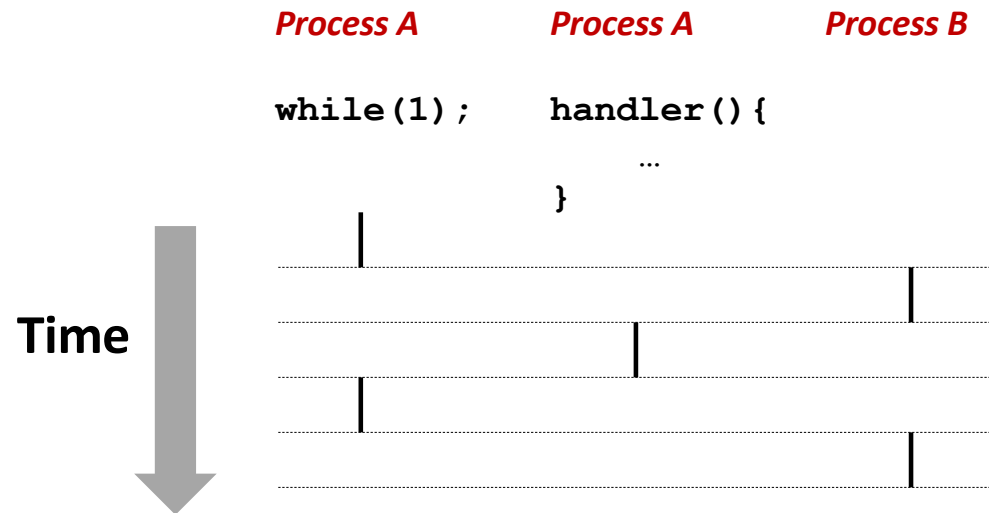
    /* Wait for the receipt of a signal */
    pause();

    return 0;
}
```

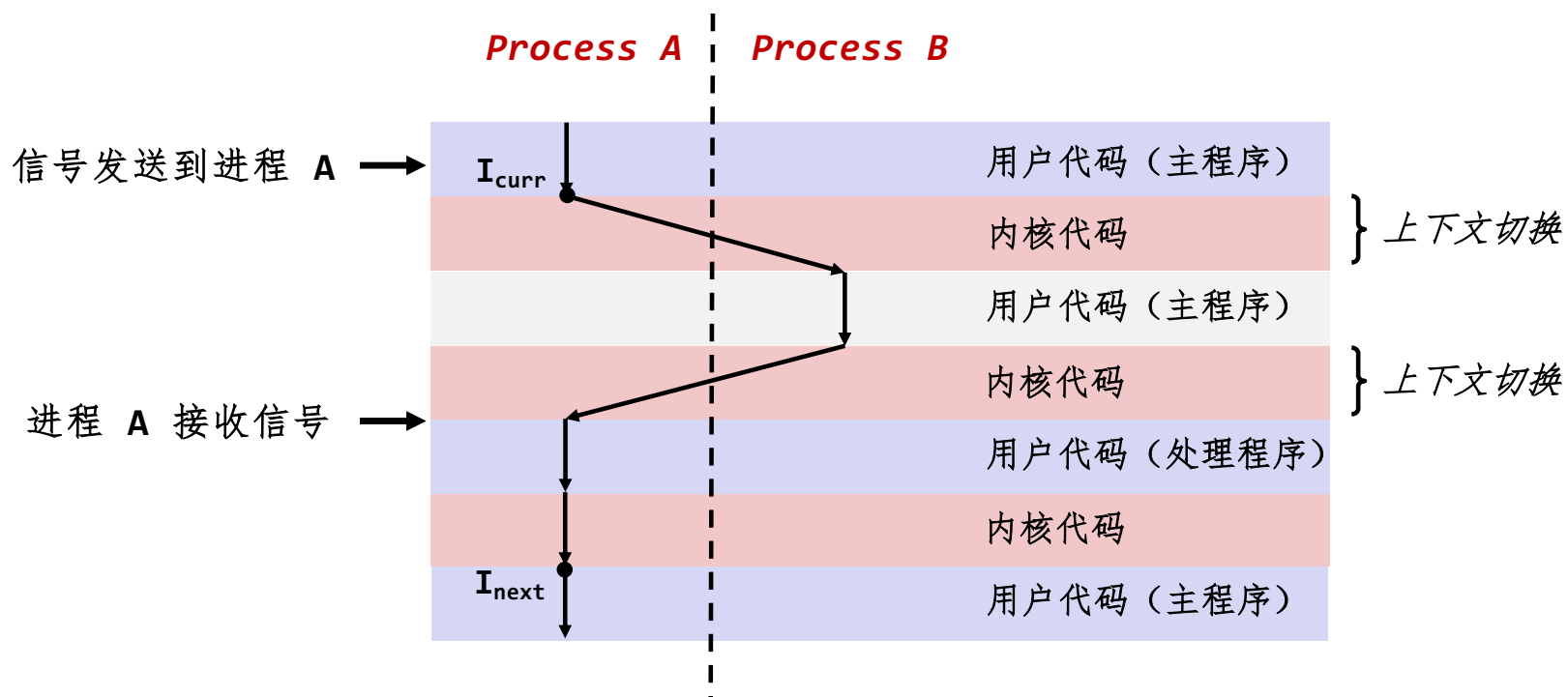
sigint.c

信号处理程序作为并发逻辑流

- 信号处理程序是一个独立的逻辑流（而不是一个进程），它与主程序并发运行

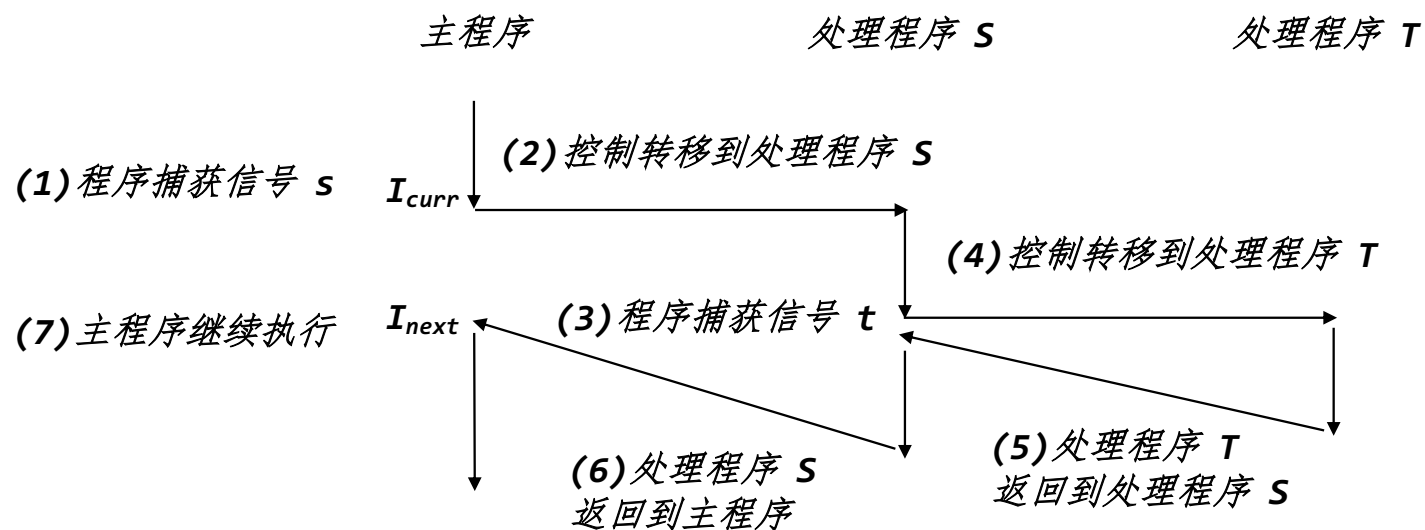


信号处理程序作为并发逻辑流的另一视角



嵌套信号处理程序

- 信号处理程序可以被其他信号处理程序中断



阻塞和解除阻塞信号

■ 隐式阻塞机制

- 内核默认阻塞任何当前处理程序正在处理信号类型的待处理信号
- 例如，一个 **SIGINT** 处理程序不会被另一个 **SIGINT** 信号中断

■ 显式阻塞机制

- `sigprocmask` 函数

■ 辅助函数

- `sigemptyset` - 初始化为空集合
- `sigfillset` - 将所有信号编号添加到集合中
- `sigaddset` - 将特定信号编号添加到集合中
- `sigdelset` - 从集合中删除特定信号编号

临时阻塞信号

```
sigset_t mask, prev_mask;

Sigemptyset(&mask);
Sigaddset(&mask, SIGINT);

/* Additionally block SIGINT and save previous blocked set */
Sigprocmask(SIG_BLOCK, &mask, &prev_mask);

    : /* Code region that will not be interrupted by SIGINT */

/* Restore previous blocked set, unblocking SIGINT */
Sigprocmask(SIG_SETMASK, &prev_mask, NULL);
```

安全的信号处理

- 信号处理程序很麻烦，因为它们与主程序并发执行，并共享相同的全局数据结构
 - 共享的数据结构可能被破坏。
- 以下是一些帮助你避免问题的原则

编写安全的处理程序的原则

- **G0: 处理程序要尽可能简单**
 - 例如，设置全局标志并立即返回
- **G1: 在处理程序中只调用异步信号安全的函数**
 - `printf`, `sprintf`, `malloc` 和 `exit` 都不安全
- **G2: 保存和恢复 `errno`**
 - 这样其他处理程序就不会覆盖你的 `errno` 值
- **G3: 阻塞所有的信号，保护对共享全局数据结构的访问**
 - 防止可能的损坏
- **G4: 用 `volatile` 声明全局变量**
 - 防止编译器将它们缓存在寄存器中
- **G5: 用 `volatile sig_atomic_t` 声明标志**
 - *flag*: 仅被读取或写入的变量 (e.g. `flag = 1`, not `flag++`)
 - 以这种方式声明的标志不需要像其他全局变量那样受到保护

异步信号安全

- **异步信号安全**函数要么是可重入的（例如，所有变量都存储在栈帧中，见 12.7.2 节），要么它不能被信号处理程序中断。
- **Posix** 保证了 **117** 个函数是异步信号安全的
 - 来源：“man 7 signal”
 - 常用的、异步信号安全函数：
 - `_exit`, `write`, `wait`, `waitpid`, `sleep`, `kill`
 - 常用的、不在异步信号安全列表中的函数：
 - `printf`, `sprintf`, `malloc`, `exit`
 - 遗憾的事实：`write` 是唯一的异步信号安全的输出函数

安全生成格式化输出

■ 在处理程序中使用 `csapp.c` 中的可重入 **SIO (Safe I/O library)**

- `ssize_t sio_puts(char s[]) /* Put string */`
- `ssize_t sio_putl(long v) /* Put long */`
- `void sio_error(char s[]) /* Put msg & exit */`

```
void sigint_handler(int sig) /* Safe SIGINT handler */
{
    Sio_puts("So you think you can stop the bomb with ctrl-c,
do you?\n");
    sleep(2);
    Sio_puts("Well...");
    sleep(1);
    Sio_puts("OK. :-)\n");
    _exit(0);
}
```

sigintsafe.c

正确的信号处理

```
int ccount = 0;
void child_handler(int sig) {
    int olderrno = errno;
    pid_t pid;
    if ((pid = wait(NULL)) < 0)
        Sio_error("wait error");
    ccount--;
    Sio_puts("Handler reaped child ");
    Sio_putl((long)pid);
    Sio_puts(" \n");
    sleep(1);
    errno = olderrno;
}

void fork14() {
    pid_t pid[N];
    int i;
    ccount = N;
    Signal(SIGCHLD, child_handler);

    for (i = 0; i < N; i++) {
        if ((pid[i] = Fork()) == 0) {
            Sleep(1);
            exit(0); /* Child exits */
        }
    }
    while (ccount > 0) /* Parent spins */
        ;
}
```

forks.c

- 未处理的信号是不排队的
 - pending 位向量中每种类型的信号只对应有一位
 - 所以每种类型最多只能有一个未处理的信号
- 不可以用信号来对其他进程中发生的事件计数，例如子进程终止的次数

```
whaleshark> ./forks 14
Handler reaped child 23240
Handler reaped child 23241
```

正确的信号处理

■ 必须等待所有终止的子进程

- 将 `wait` 放在循环中，以收割所有已终止的子进程

```
void child_handler2(int sig)
{
    int olderrno = errno;
    pid_t pid;
    while ((pid = wait(NULL)) > 0) {
        ccount--;
        Sio_puts("Handler reaped child ");
        Sio_putl((long)pid);
        Sio_puts(" \n");
    }
    if (errno != ECHILD)
        Sio_error("wait error");
    errno = olderrno;
}
```

```
whaleshark> ./forks 15
Handler reaped child 23246
Handler reaped child 23247
Handler reaped child 23248
Handler reaped child 23249
Handler reaped child 23250
whaleshark>
```

同步流以避免竞争

```
int main(int argc, char **argv)
{
    int pid;
    sigset_t mask_all, prev_all;

    Sigfillset(&mask_all);
    Signal(SIGCHLD, handler);
    initjobs(); /* Initialize the job list */

    while (1) {
        if ((pid = Fork()) == 0) { /* Child */
            Execve("/bin/date", argv, NULL);
        }
        Sigprocmask(SIG_BLOCK, &mask_all, &prev_all); /* Parent */
        addjob(pid); /* Add the child to the job list */
        Sigprocmask(SIG_SETMASK, &prev_all, NULL);
    }
    exit(0);
}
```

procmask1.c

同步流以避免竞争

■ 简单 Shell 的 SIGCHLD 处理程序

```
void handler(int sig)
{
    int olderrno = errno;
    sigset_t mask_all, prev_all;
    pid_t pid;

    Sigfillset(&mask_all);
    while ((pid = waitpid(-1, NULL, 0)) > 0) { /* Reap child */
        Sigprocmask(SIG_BLOCK, &mask_all, &prev_all);
        deletejob(pid); /* Delete the child from the job list */
        Sigprocmask(SIG_SETMASK, &prev_all, NULL);
    }
    if (errno != ECHILD)
        Sio_error("waitpid error");
    errno = olderrno;
}
```

procmask1.c

同步流以避免竞争

- 这个简单 Shell 有不易察觉的同步错误，因为它假设父进程在子进程之前运行

```
int main(int argc, char **argv)
{
    int pid;
    sigset_t mask_all, prev_all;

    Sigfillset(&mask_all);
    Signal(SIGCHLD, handler);
    initjobs(); /* Initialize the job list */

    while (1) {
        if ((pid = Fork()) == 0) { /* Child */
            Execve("/bin/date", argv, NULL);
        }
        Sigprocmask(SIG_BLOCK, &mask_all, &prev_all); /* Parent */
        addjob(pid); /* Add the child to the job list */
        Sigprocmask(SIG_SETMASK, &prev_all, NULL);
    }
    exit(0);
}
```

procmask1.c

没有竞争的正确 Shell 程序

```
int main(int argc, char **argv)
{
    int pid;
    sigset_t mask_all, mask_one, prev_one;

    Sigfillset(&mask_all);
    Sigemptyset(&mask_one);
    Sigaddset(&mask_one, SIGCHLD);
    Signal(SIGCHLD, handler);
    initjobs(); /* Initialize the job list */

    while (1) {
        Sigprocmask(SIG_BLOCK, &mask_one, &prev_one); /* Block SIGCHLD */
        if ((pid = Fork()) == 0) { /* Child process */
            Sigprocmask(SIG_SETMASK, &prev_one, NULL); /* Unblock SIGCHLD */
            Execve("/bin/date", argv, NULL);
        }
        Sigprocmask(SIG_BLOCK, &mask_all, NULL); /* Parent process */
        addjob(pid); /* Add the child to the job list */
        Sigprocmask(SIG_SETMASK, &prev_one, NULL); /* Restore signal processing */
    }
    exit(0);
}
```

procmask2.c

Before: 简单的 Shell eval 函数

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE];    /* Holds modified command line */
    int bg;               /* Should the job run in bg or fg? */
    pid_t pid;            /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */
        if (!bg) {
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

shellex.c

显式地等待信号

■ 处理程序显式等待 SIGCHLD 的到来

```
volatile sig_atomic_t pid;

void sigchld_handler(int s)
{
    int olderrno = errno;
    pid = Waitpid(-1, NULL, 0); /* Main is waiting for nonzero pid */
    errno = olderrno;
}

void sigint_handler(int s)
{
}
```

waitforsignal.c

显式地等待信号

类似于等待前台作业终止的 Shell

```
int main(int argc, char **argv) {
    sigset_t mask, prev;
    Signal(SIGCHLD, sigchld_handler);
    Signal(SIGINT, sigint_handler);
    Sigemptyset(&mask);
    Sigaddset(&mask, SIGCHLD);

    while (1) {
        Sigprocmask(SIG_BLOCK, &mask, &prev); /* Block SIGCHLD */
        if (Fork() == 0) /* Child */
            exit(0);
        /* Parent */
        pid = 0;
        Sigprocmask(SIG_SETMASK, &prev, NULL); /* Unblock SIGCHLD */

        /* Wait for SIGCHLD to be received (wasteful!) */
        while (!pid)
            ;
        /* Do some work after receiving SIGCHLD */
        printf(".");
    }
    exit(0);
}
```

waitforsignal.c

显式地等待信号

- 程序是正确的，但非常浪费

- 程序进入忙等待循环

```
while (!pid)
    ;
```

- 可能有竞争条件

- 在检查 pid 后和 pause 之前，可能会收到信号

```
while (!pid) /* Race! */
    pause();
```

- 安全，但太慢

- 需要一秒才能响应

```
while (!pid) /* Too slow! */
    sleep(1);
```

使用 `sigsuspend`

- `int sigsuspend(const sigset_t *mask)`

- 下面代码的相当于原子（不可中断）版本：

```
sigprocmask(SIG_BLOCK, &mask, &prev);  
pause();  
sigprocmask(SIG_SETMASK, &prev, NULL);
```

使用 sigsuspend

```
int main(int argc, char **argv) {
    sigset_t mask, prev;
    Signal(SIGCHLD, sigchld_handler);
    Signal(SIGINT, sigint_handler);
    Sigemptyset(&mask);
    Sigaddset(&mask, SIGCHLD);

    while (1) {
        Sigprocmask(SIG_BLOCK, &mask, &prev); /* Block SIGCHLD */
        if (Fork() == 0) /* Child */
            exit(0);

        /* Wait for SIGCHLD to be received */
        pid = 0;
        while (!pid)
            Sigsuspend(&prev);

        /* Optionally unblock SIGCHLD */
        Sigprocmask(SIG_SETMASK, &prev, NULL);
        /* Do some work after receiving SIGCHLD */
        printf(".");
    }
    exit(0);
}
```

sigsuspend.c

总结

■ 信号提供进程级别的异常处理

- 可以从用户程序中产生
- 可以通过声明信号处理程序来定义信号的作用
- 小心地编写信号处理程序

■ 非局部跳转提供进程内的异常控制流

- 遵循栈规则的约束